

cw — parameter study

Generated by `sweep/analyze_sweep.py`

2026-08-04

Setup

458 configurations of `./cw`, each run at 5 seeds (42, 43, 44, 45, 46) of 200 instances, so 1,000 instances per configuration. Instances are generated with the Kool/NeuOpt law (coordinates $U[0,1]^2$, demands $U\{1, \dots, 9\}$, capacity from `default_capacity`). Unless a study says otherwise, $n = 100$ and `--sa-steps` = 10,000,000.

Every comparison is paired. Instance k is generated from `seed + k` (`cw.c:2574`), so all runs of a study see byte-identical instances. Rather than comparing two means with their independent standard deviations, we compare $\text{cost}_i(A)$ with $\text{cost}_i(B)$ on the same instance i and report the mean of the 1,000 differences with a 95 % confidence interval, plus a distribution-free sign test. This matters: the between-instance spread ($\sigma \approx 1.9$ at $n = 100$) is more than an order of magnitude larger than the effects being measured.

Every configuration is replicated on 5 seeds. `cw` has a single `--seed`, and it drives both the instance set (`cw.c:2574`) and the annealing RNG (`cw.c:2679`); there is no way to re-randomise the solver while holding the instances fixed. A seed is therefore a complete replication — fresh instances, fresh randomness — and the 5 blocks are concatenated before the paired statistics are taken. No conclusion in this report rests on a single instance draw, and each Δ is the mean of 1,000 paired differences rather than 200.

Everything is measured at two budgets. Each grid is run at `--sa-steps` = 10^5 and again at 10^7 , the budget this solver is actually run at. Section 2 puts the two side by side; the body of the report is the 10^7 -step tier.

In every table, Δ is that paired mean as a percentage of the baseline; **bold** means the CI excludes zero *and* the sign test rejects at 5%; **win/loss** counts instances strictly improved and strictly worsened. **Negative is better** — these are costs.

Contents

1	Overview: what each knob is worth	2
2	Does any of this survive a 100× larger budget?	3
3	Initialisation: where the annealing starts from	5

4	Move operators	8
5	swap* and route opening	11
6	Candidate neighbourhood	14
7	Compute cost	15
8	Restarts	17
9	Optimal Split	19
10	Vertex selection	21
11	Biasing the second vertex	24
12	Racing between restarts, and interleaving	27
13	Annealing schedule	30
14	Construction quality	32
15	Do the knobs combine?	35
16	The configuration to use	37
16.1	The command	38
16.2	What the changed options do	39
16.3	Confirmation	39
17	Caveats	40

1 Overview: what each knob is worth

Each row below varies one knob with everything else left at its default, and reports the best setting the sweep found for it. Negative is better. The sections that follow explain what each knob does and give the full grids.

Read the confidence intervals, not the point estimates. A setting is listed *because* it came out lowest, so its Δ is optimistically biased — the more values a row sweeps, the stronger the bias. A row whose CI straddles zero is a knob that did nothing here. This is why Section 16 re-runs the winners on a seed the sweep never saw.

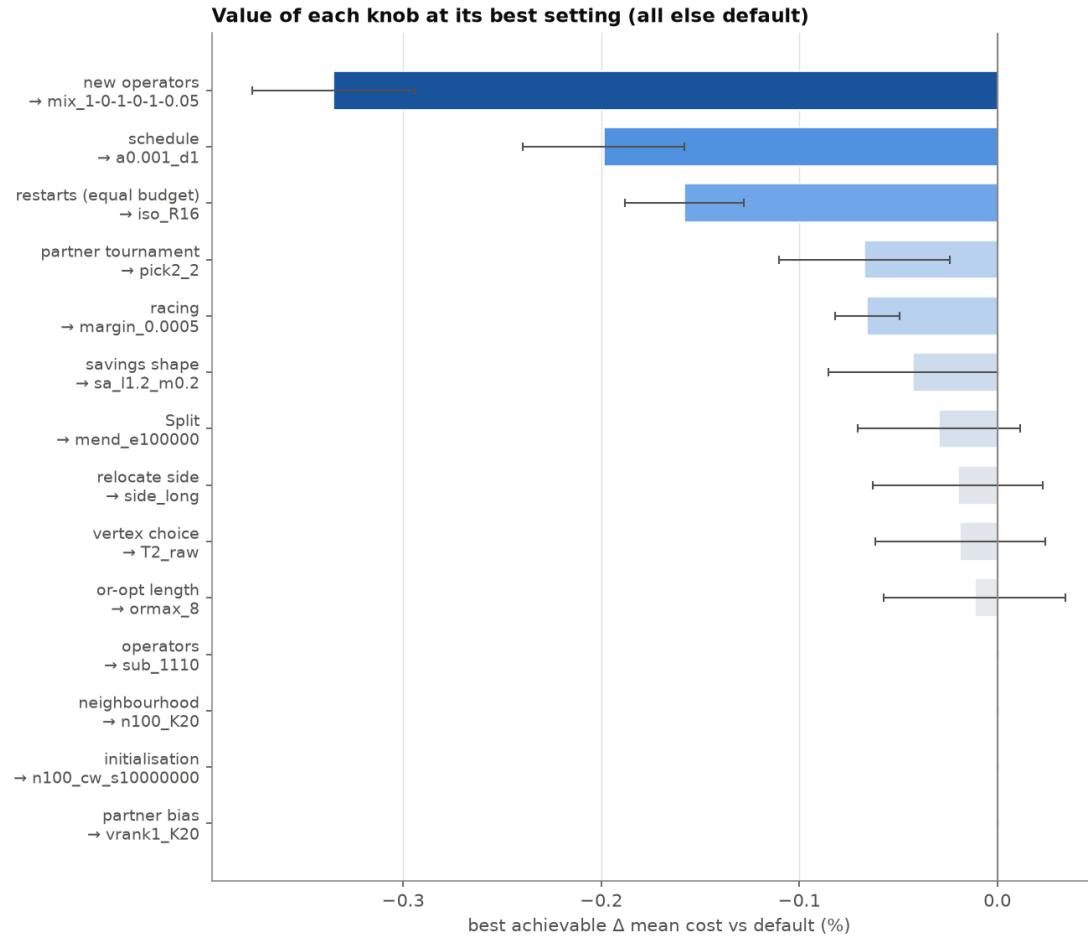


Figure 1: Best achievable improvement per knob, with 95 % CIs. Bars that cross zero are knobs with no measurable effect on this instance family at this budget.

knob	best setting found	Δ (%)	95 % CI	win/loss
new operators	<code>mix_1-0-1-0-1-0.05</code>	-0.335	[-0.376, -0.294]	692/278
schedule	<code>a0.001_d1</code>	-0.199	[-0.240, -0.158]	607/356
restarts (equal budget)	<code>iso_R16</code>	-0.158	[-0.188, -0.128]	562/389
partner tournament	<code>pick2_2</code>	-0.067	[-0.110, -0.024]	536/437
racing	<code>margin_0.0005</code>	-0.066	[-0.082, -0.049]	494/293
savings shape	<code>sa_l1.2_m0.2</code>	-0.042	[-0.085, +0.000]	504/440
Split	<code>mend_e100000</code>	-0.030	[-0.071, +0.012]	492/467
relocate side	<code>side_long</code>	-0.020	[-0.063, +0.023]	504/467
vertex choice	<code>T2_raw</code>	-0.019	[-0.062, +0.024]	496/473
or-opt length	<code>ormax_8</code>	-0.011	[-0.057, +0.035]	513/466
operators	<code>sub_1110</code>	+0.000	[+0.000, +0.000]	0/0
neighbourhood	<code>n100_K20</code>	+0.000	[+0.000, +0.000]	0/0
initialisation	<code>n100_cw_s10000000</code>	+0.000	[+0.000, +0.000]	0/0
partner bias	<code>vrank1_K20</code>	+0.000	[+0.000, +0.000]	0/0

Table 1: Each knob at its best setting, paired against its own default.

2 Does any of this survive a $100\times$ larger budget?

Every grid in this report is run twice: once at `--sa-steps = 100,000` with 1000 instances per seed, and once at 10,000,000 with 200. A knob tuned at the lower budget is only useful if its answer still holds at the higher one, which is where this solver is actually run — and that is not something a single-budget sweep can check. The two columns below are the same measurement at the two budgets.

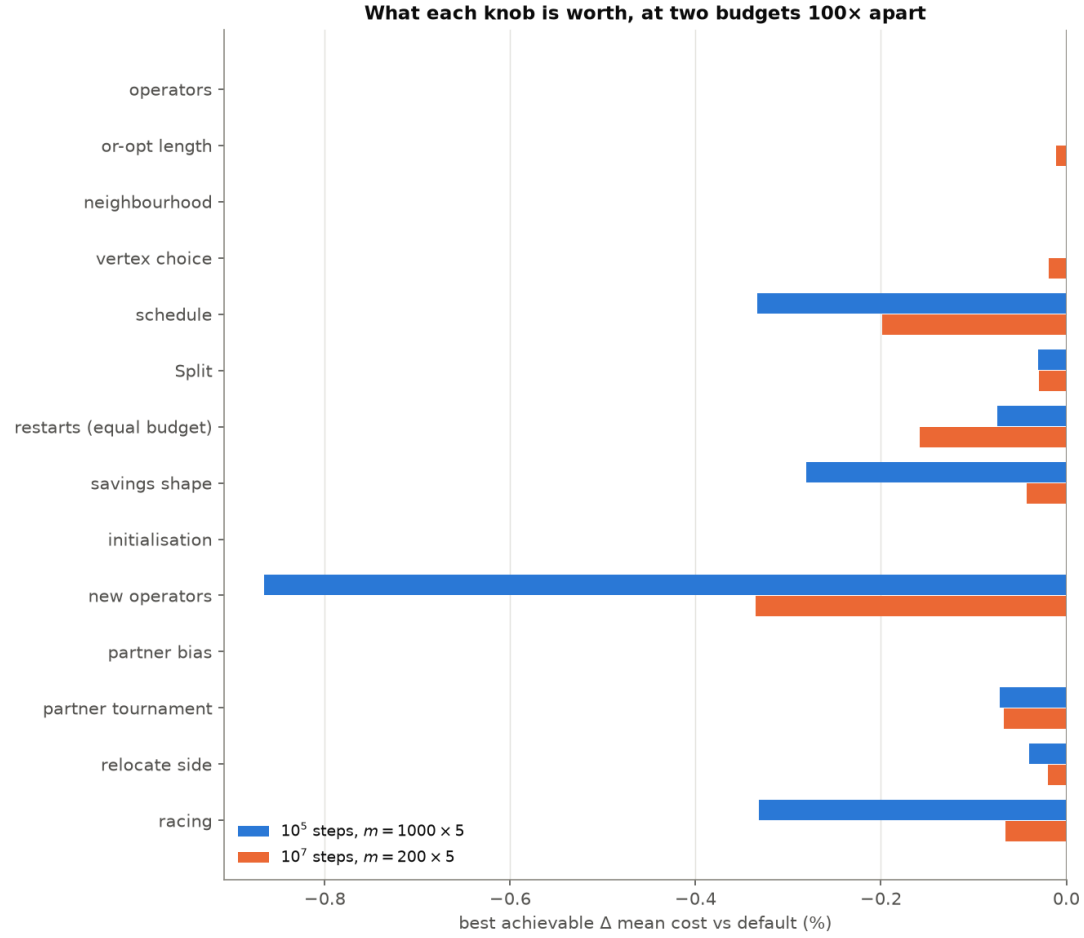


Figure 2: Each knob’s best achievable improvement, measured independently at the two budgets. A bar pair that changes sign or magnitude is a recommendation that does not transfer.

Table 2: Each knob at each budget, with both intervals — they are not equally precise, since m is 1000 per seed at 10^5 and 200 at 10^7 . *stable* means the same setting won and stayed significant; *inert at both* means the knob never earned a change. Anything else is a recommendation that depends on the budget.

knob	best @ 10^5	Δ (%)	95 % CI	best @ 10^7	Δ (%)	95 % CI	verdict
operators	sub_1110	+0.000	[+0.000, +0.000]	sub_1110	+0.000	[+0.000, +0.000]	inert at both
or-opt length	ormax_3	+0.000	[+0.000, +0.000]	ormax_8	-0.011	[-0.057, +0.035]	inert at both
neighbourhood	n100_K20	+0.000	[+0.000, +0.000]	n100_K20	+0.000	[+0.000, +0.000]	inert at both
vertex choice	T2_lb	+0.000	[+0.000, +0.000]	T2_raw	-0.019	[-0.062, +0.024]	inert at both
schedule	a0.001_d1	-0.333	[-0.365, -0.302]	a0.001_d1	-0.199	[-0.240, -0.158]	stable
Split	mend_e10000	-0.030	[-0.055, -0.006]	mend_e100000	-0.030	[-0.071, +0.012]	significant only at 10^5
restarts (equal budget)	iso_R8	-0.075	[-0.106, -0.043]	iso_R16	-0.158	[-0.188, -0.128]	significant only at 10^7
savings shape	sa_l1.4_m0.5	-0.280	[-0.321, -0.240]	sa_l1.2_m0.2	-0.042	[-0.085, +0.000]	significant only at 10^5
initialisation	n100_cw_s100000	+0.000	[+0.000, +0.000]	n100_cw_s10000000	+0.000	[+0.000, +0.000]	inert at both
new operators	mix_1-0-1-0-1-0.05	-0.865	[-0.898, -0.832]	mix_1-0-1-0-1-0.05	-0.335	[-0.376, -0.294]	stable
partner bias	vrank1_K20	+0.000	[+0.000, +0.000]	vrank1_K20	+0.000	[+0.000, +0.000]	inert at both
partner tournament	pick2_2	-0.072	[-0.103, -0.041]	pick2_2	-0.067	[-0.110, -0.024]	stable
relocate side	side_long	-0.040	[-0.071, -0.009]	side_long	-0.020	[-0.063, +0.023]	significant only at 10^5
racing	margin_0.0005	-0.332	[-0.350, -0.313]	margin_0.0005	-0.066	[-0.082, -0.049]	stable

4 of 14 knobs do not transfer: Split, restarts (equal budget), savings shape, relocate side. For these the setting recommended in Section 16 is the one measured at 10^7 steps, because that is the budget this solver is run at; the 10^5 column is kept only to show that the disagreement exists.

3 Initialisation: where the annealing starts from

What it controls. `--init` chooses the solution the simulated annealing starts from. `cw` (the default) runs the Clarke & Wright savings construction: every pair of customers gets a score $s(i, j) = d_{0i} + d_{0j} - \lambda d_{ij} + \mu |d_{0i} - d_{0j}|$, measuring how much is saved by serving i and j on one route instead of two, and merges route endpoints greedily in decreasing score while capacity allows. `random` throws that away: it shuffles the customers and cuts the permutation into routes at the capacity limit (first fit). Under `--restarts` each restart draws its own permutation.

The question. A random start is 100–380 % worse before annealing. Does the annealing erase that, and how much budget does it cost to do so?

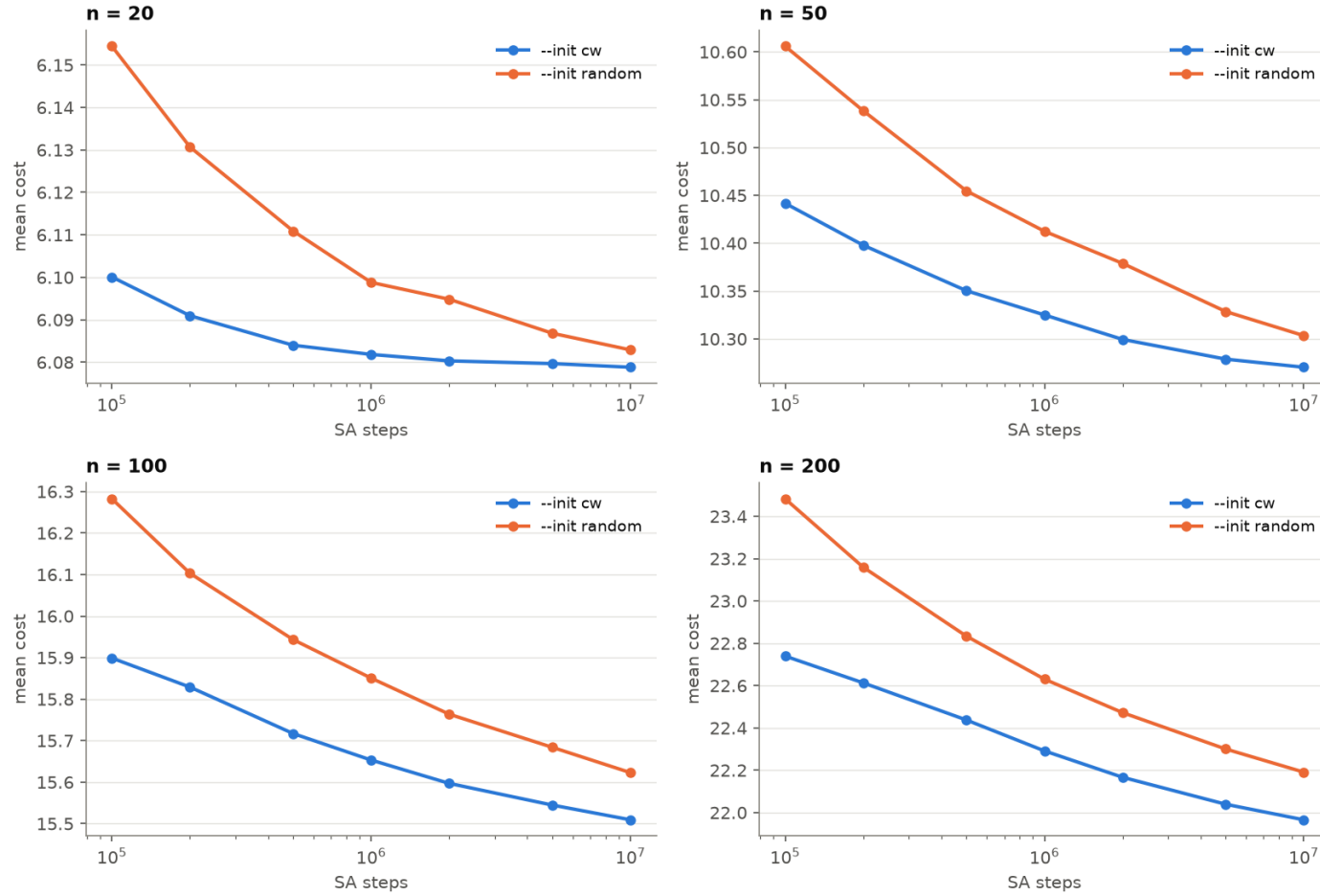


Figure 3: Mean cost against the annealing budget, for both initialisations, at four dimensions. Same budget on both sides of each panel.

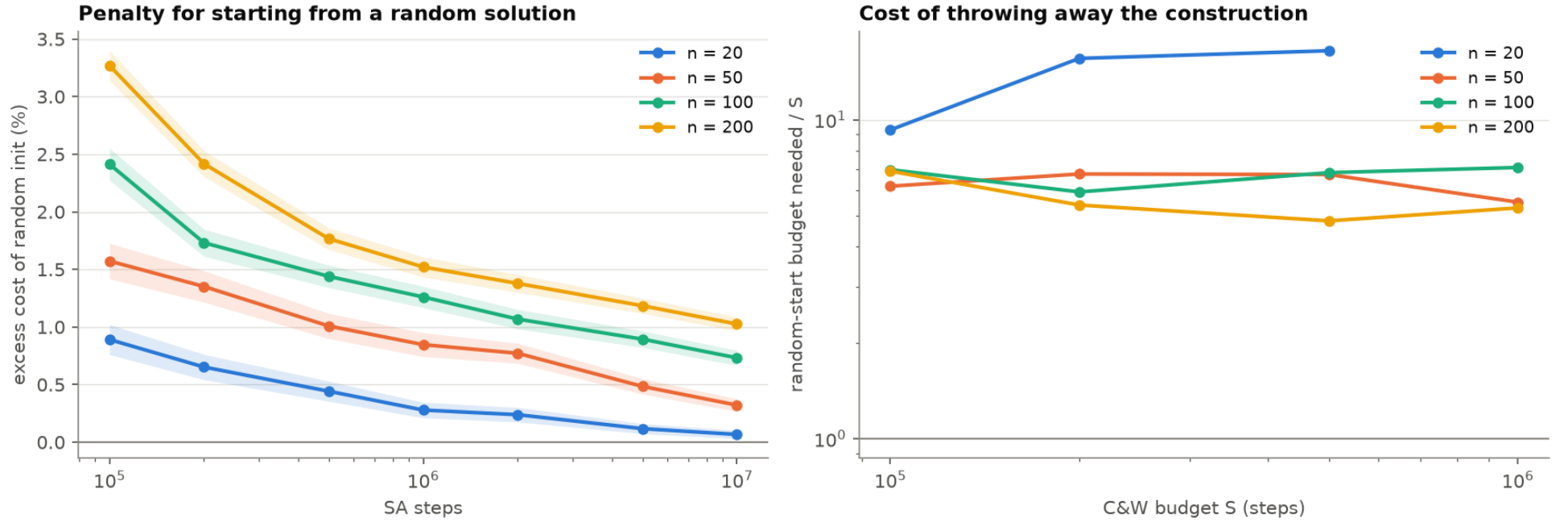


Figure 4: Left: paired excess of the random start over Clarke & Wright at equal budget, with a 95 % band. Right: how many steps the random start needs to reach what Clarke & Wright reaches at S steps.

n	excess at 100,000 (%)	excess at 10,000,000 (%)	steps to catch up
20	+0.89	+0.07	not within 10,000,000
50	+1.57	+0.32	not within 10,000,000
100	+2.41	+0.73	not within 10,000,000
200	+3.27	+1.03	not within 10,000,000

Table 3: Penalty for a random start, paired against the C&W start at the same budget. ‘Catch up’ is the first budget at which the upper end of the 95 % CI reaches zero.

n	at smallest S	mid-range S	at largest S
20	9×	16×	16×
50	6×	7×	6×
100	7×	7×	7×
200	7×	5×	5×

Table 4: Budget multiplier: steps the random start needs to match the C&W start, relative to the C&W budget. A ratio of 1 means the head start has been fully repaid.

Reading. The random start converges towards parity as the budget grows: the penalty falls from +0.89 % at 100,000 steps to +0.07 % at 10,000,000. It never overtakes — not one of the 28 cells in the grid has the random start significantly ahead — so it buys no diversity the annealing can exploit. Keep the construction.

4 Move operators

What it controls. `--ops r,s,t,o` sets the relative probabilities of the four neighbourhood moves the annealing draws from. Each move first picks a vertex u (Section 10) and a partner v near it (Section 6), then:

- **relocate** (`mv_relocate`, `cw.c:1040`) — take u out of its route and re-insert it next to v , possibly in another route.
- **swap** (`mv_swap`, `cw.c:1083`) — exchange the positions of u and v .
- **2-opt** (`mv_2opt`, `cw.c:1362`) — replace the two edges carried by u and v by the other pairing; inside one route this reverses a segment, across two routes it exchanges their tails.
- **or-opt** (`mv_oropt`, `cw.c:1148`) — move a whole run of 2 to `--or-max` consecutive customers starting at u , optionally reversed, next to v .

Every move is accepted by the Metropolis rule on its exact cost delta. The default is `1,1,1,0`: or-opt is present in the code but switched off. `--ops` takes two further weights, for `swap*` and route opening, which are also zero by default and have a section of their own (Section 5); this section is about the original four, so every run in it leaves the other two at zero.

The question. Is the default subset the right one, is the uniform weighting right, and is or-opt off for a good reason?

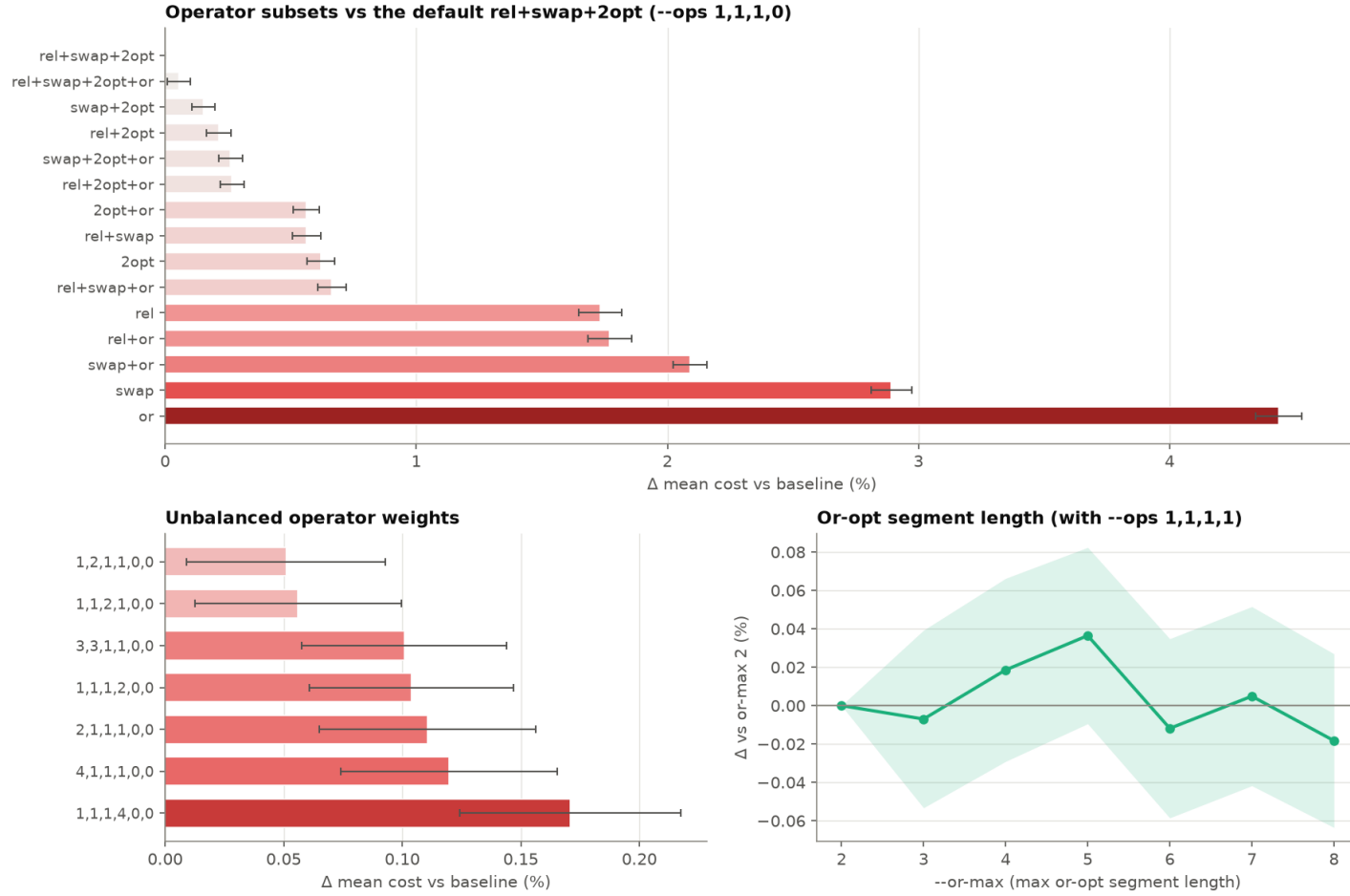


Figure 5: Top: all 15 non-empty operator subsets. Bottom left: unbalanced weightings. Bottom right: the or-opt segment length cap. Bars are paired deltas against the default with 95 % CIs.

subset	mean cost	Δ (%)	95 % CI	win/loss
rel+swap+2opt	15.50934	+0.000	[+0.000, +0.000]	0/0
rel+swap+2opt+or	15.51793	+0.055	[+0.010, +0.101]	471/511
swap+2opt	15.53289	+0.152	[+0.105, +0.198]	418/572
rel+2opt	15.54226	+0.212	[+0.163, +0.261]	395/580
swap+2opt+or	15.54971	+0.260	[+0.214, +0.307]	354/641
rel+2opt+or	15.55068	+0.267	[+0.219, +0.314]	367/614
2opt+or	15.59656	+0.562	[+0.511, +0.613]	228/767
rel+swap	15.59669	+0.563	[+0.507, +0.620]	242/756
2opt	15.60547	+0.620	[+0.566, +0.674]	227/769
rel+swap+or	15.61235	+0.664	[+0.607, +0.721]	203/792
rel	15.77827	+1.734	[+1.648, +1.820]	63/936
rel+or	15.78391	+1.770	[+1.683, +1.857]	62/937
swap+or	15.83347	+2.090	[+2.024, +2.156]	13/987
swap	15.95790	+2.892	[+2.812, +2.973]	6/994
or	16.19698	+4.434	[+4.341, +4.526]	0/1000

Table 5: All 15 non-empty operator subsets, paired against the default `--ops 1,1,1,0`. Bold = significant at 95 % and by sign test.

weights	mean cost	Δ (%)	95 % CI	win/loss
1,2,1,1,0,0	15.51725	+0.051	[+0.009, +0.093]	460/502
1,1,2,1,0,0	15.51804	+0.056	[+0.012, +0.100]	464/509
3,3,1,1,0,0	15.52499	+0.101	[+0.058, +0.144]	425/547
1,1,1,2,0,0	15.52545	+0.104	[+0.061, +0.147]	418/562
2,1,1,1,0,0	15.52649	+0.111	[+0.065, +0.156]	436/543
4,1,1,1,0,0	15.52790	+0.120	[+0.074, +0.165]	441/536
1,1,1,4,0,0	15.53585	+0.171	[+0.124, +0.218]	387/593

Table 6: Unbalanced weightings, against the same default.

--or-max	mean cost	Δ (%)	95 % CI	win/loss
2	15.51901	+0.000	[+0.000, +0.000]	0/0
3	15.51793	-0.007	[-0.053, +0.039]	490/493
4	15.52190	+0.019	[-0.029, +0.066]	486/484
5	15.52467	+0.036	[-0.009, +0.082]	473/507
6	15.51717	-0.012	[-0.058, +0.035]	500/472
7	15.51977	+0.005	[-0.042, +0.052]	483/495
8	15.51618	-0.018	[-0.063, +0.027]	507/460

Table 7: Or-opt segment length, with or-opt enabled (`--ops 1,1,1,1`). Valid range is 2–8 (`cw.c:2546`).

Reading. The default subset wins: every other subset and weighting is worse or indistinguishable, and turning or-opt on costs a small but significant amount. Or-opt overlaps with relocate (a length-1 relocate is the same move) while being more expensive per draw, so at a fixed step count it buys less. `--or-max` is then irrelevant — its whole range sits inside the noise. Leaving or-opt off is the right default.

5 swap* and route opening

What it controls. `--ops` carries two further weights beyond the four of Section 4, both defaulting to zero:

- **swap*** (`mv_swapstar`, `cw.c:1216`) — Vidal’s *Hybrid genetic search for the CVRP*, C&OR 2022. Ordinary swap forces u into v ’s slot; swap* drops that and re-inserts each of the two customers at its *best* position in the other’s route. The two routes are disjoint, so the two re-insertions are independent and the delta is still exact — but the scan costs $O(L_1 + L_2)$ instead of $O(1)$. The kNN neighbour selects the target *route*, not the partner: v is then the worse by regret of two uniform customers of that route.
- **opening** (`mv_open`, `cw.c:1325`) — isolate one customer in an empty route. Almost always worsening, and that is the point: the annealing can empty a route but has no move that repopulates one, so the number of active routes is a one-way door between two Splits. This is the only move that opens it again.

The question. swap* is the one non-elementary operator in the solver, so it has to earn its cost: at a fixed step count it is not competing on equal terms. Everything below is therefore also measured against wall time.

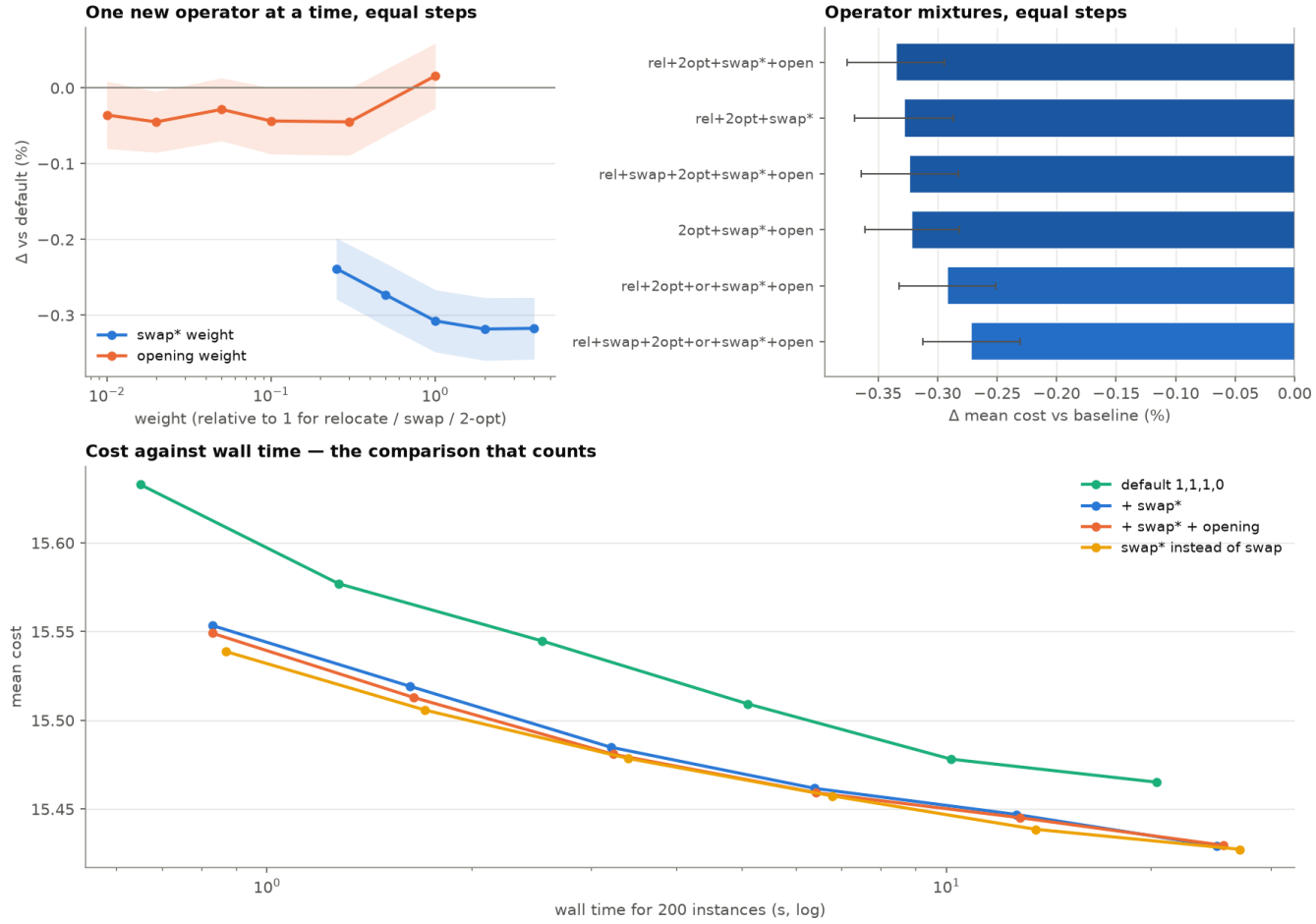


Figure 6: Top left: each new operator's weight swept alone. Top right: mixtures. Bottom: cost against measured wall time for four configurations swept over a 32x budget range — a curve that sits lower-left dominates.

mixture	mean cost	Δ (%)	95 % CI	win/loss
rel+2opt+swap*+open	15.45738	-0.335	[-0.376, -0.294]	692/278
rel+2opt+swap*	15.45844	-0.328	[-0.370, -0.287]	708/268
rel+swap+2opt+swap*+open	15.45920	-0.323	[-0.364, -0.282]	710/265
2opt+swap*+open	15.45947	-0.322	[-0.361, -0.282]	700/288
rel+2opt+or+swap*+open	15.46411	-0.292	[-0.332, -0.251]	673/303
rel+swap+2opt+or+swap*+open	15.46722	-0.272	[-0.313, -0.231]	676/298

Table 8: Operator mixtures at equal step count, paired against the stock `--ops 1,1,1,0,0,0`. Equal steps flattens swap*, which is why the wall-time reading below is the one to trust.

configuration	time	default	this	Δ	time	default	this	Δ
+ swap*	5.09s	15.50934	15.46933	-0.258 %	20.40s	15.46515	15.43449	-0.198 %
+ swap* + opening	5.09s	15.50934	15.46656	-0.276 %	20.40s	15.46515	15.43467	-0.197 %
swap* instead of swap	5.09s	15.50934	15.46620	-0.278 %	20.40s	15.46515	15.43191	-0.215 %

Table 9: Iso-time reading of the bottom panel: both curves interpolated in $\log(\text{wall time})$ at two points of the default’s own ladder. Negative Δ = better at the same wall time.

n	default	+swap*+open	Δ (%)	95 % CI	win/loss	time x
20	6.07883	6.07638	-0.040	[-0.056, -0.025]	46/0	1.21
50	10.27040	10.25748	-0.126	[-0.154, -0.097]	434/192	1.23
100	15.50934	15.45920	-0.323	[-0.364, -0.282]	710/265	1.27
200	21.96580	21.85268	-0.515	[-0.558, -0.472]	792/208	1.34
500	37.14129	36.69422	-1.204	[-1.251, -1.157]	980/20	1.46

Table 10: `--ops 1,1,1,0,1,0.05` against the default across dimension, at equal step count. The last column is the wall time ratio — the price of the equal-step gain.

Reading. swap* is a clear improvement at equal step count. It costs $1.27\times$ the wall time per step at $n = 100$, and the iso-time table still shows it ahead — so the extra work per draw is bought back. Its weight is insensitive over the whole range tried: a quarter to four times the weight of relocate spans only 0.080 %, which is what one wants from a knob.

Route opening is a different kind of thing. On its own it is worth almost nothing, and at large weight it is actively harmful — unsurprising, since it never improves a solution by itself. It is a pure enabler: the annealing can empty a route but has no other move that repopulates one, so without it the route count is monotone between two Splits. Kept at a small weight it is close to free and slightly positive.

The one result here that contradicts the intuition behind the code is that *dropping* swap in favour of swap* is not a trade-off at all: it is the best configuration on the ladder at both time points. For any given pair, the in-place exchange is one of the positions swap* already sweeps, so $\Delta_{\text{swap}^*} \leq \Delta_{\text{swap}}$ always; the measurement says

that the ten-fold cheaper draw of ordinary swap does not buy back that domination. swap is nevertheless kept in the code as a separate operator — it is a weighting decision, and `--ops 1,1,1,0,1,0.05` remains the safer recommendation at small n , where the two are within noise of each other.

6 Candidate neighbourhood

What it controls. Once a move has picked the vertex u , it needs a partner v . `sa_cand` (`cw.c:1010`) draws v uniformly from the K nearest neighbours of u , with $K = \text{--sa-knn}$; at $K = 0$ it draws uniformly from all customers. This is what keeps a move geometrically local, so that the cost delta has a chance of being negative. K is clamped to $n - 1$ (`cw.c:1675`).

K does double duty: at $K = 0$ the kNN lists are not built, and `cw.c:1676` then *silently* forces uniform vertex selection as well, whatever `--pick` says. So $K = 0$ disables two mechanisms, not one — see Section 10.

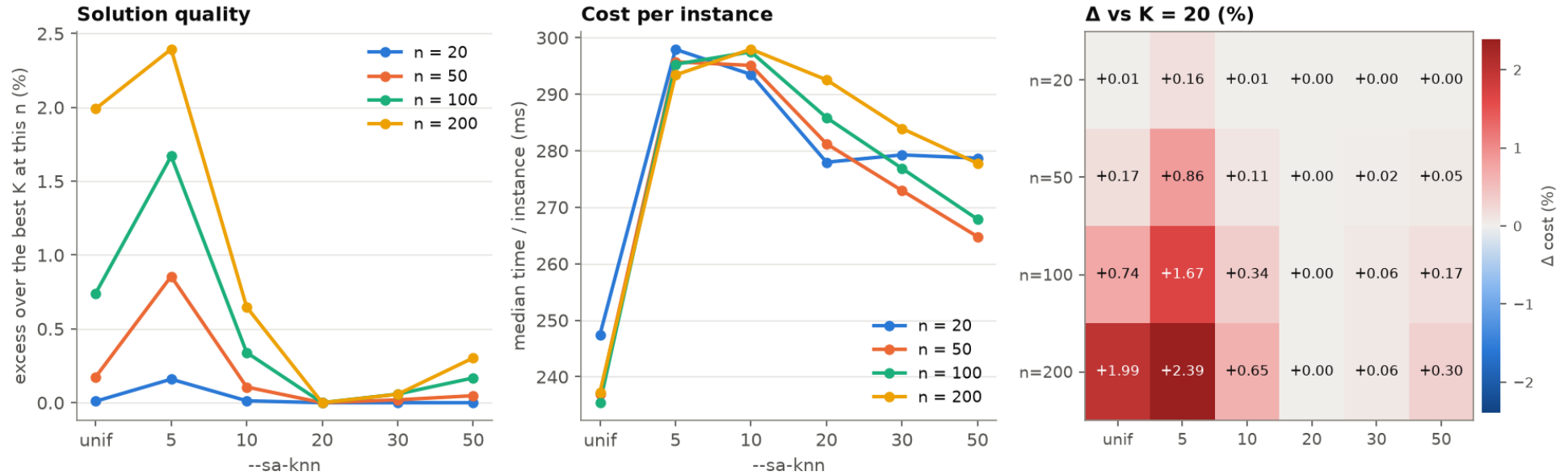


Figure 7: Quality (left, as excess over the best K at that n), cost per instance (middle) and the paired delta against $K = 20$ (right).

Table 11: `--sa-knn` across dimension, paired against $K = 20$. The effective K is $\min(K, n - 1)$, which is why the rows for $n = 20$ at $K \geq 20$ are exactly identical runs.

n	K	effective K	mean cost	Δ (%)	95 % CI	win/loss	ms/inst	
20	0	uniform	6.07942	+0.010	[+0.000, +0.019]	7/19	247.462	
20	5	5	6.08851	+0.159	[+0.107, +0.212]	41/153	297.939	
20	10	10	6.07962	+0.013	[-0.011, +0.037]	34/26	293.512	
20	20	19	6.07883	+0.000	[+0.000, +0.000]	0/0	277.966	★
20	30	19	6.07883	+0.000	[+0.000, +0.000]	0/0	279.270	
20	50	19	6.07883	+0.000	[+0.000, +0.000]	0/0	278.685	
50	0	uniform	10.28831	+0.174	[+0.133, +0.215]	271/476	236.931	
50	5	5	10.35831	+0.856	[+0.784, +0.928]	126/751	295.731	
50	10	10	10.28130	+0.106	[+0.069, +0.144]	305/407	295.104	
50	20	20	10.27040	+0.000	[+0.000, +0.000]	0/0	281.238	★
50	30	30	10.27238	+0.019	[-0.011, +0.050]	319/350	272.958	
50	50	49	10.27527	+0.047	[+0.013, +0.082]	297/385	264.797	
100	0	uniform	15.62428	+0.741	[+0.685, +0.797]	194/801	235.412	
100	5	5	15.76828	+1.670	[+1.596, +1.743]	60/940	295.308	
100	10	10	15.56204	+0.340	[+0.288, +0.391]	329/657	297.494	
100	20	20	15.50934	+0.000	[+0.000, +0.000]	0/0	285.866	★
100	30	30	15.51840	+0.058	[+0.012, +0.105]	456/526	276.874	
100	50	50	15.53535	+0.168	[+0.118, +0.217]	405/574	267.878	
200	0	uniform	22.40317	+1.991	[+1.929, +2.053]	14/986	237.255	
200	5	5	22.49174	+2.394	[+2.328, +2.461]	7/993	293.411	
200	10	10	22.10810	+0.648	[+0.600, +0.696]	191/809	297.976	
200	20	20	21.96580	+0.000	[+0.000, +0.000]	0/0	292.551	★
200	30	30	21.97839	+0.057	[+0.015, +0.100]	469/531	283.944	
200	50	50	22.03239	+0.303	[+0.257, +0.350]	342/658	277.770	

Reading. $K = 20$ is the optimum at every dimension tested ($n=20$, $n=50$, $n=100$, $n=200$), and the curve is a clean U: too small a list starves the move of good partners, too large a one dilutes it with distant ones that will be rejected. The default needs no change — and notably it does not need to grow with n .

7 Compute cost

What is measured. Two phases: the Clarke & Wright construction, which builds a savings list ($O(n^2)$ exactly, for $n \leq 1500$) and sorts it, and the annealing, which does a fixed number of $O(1)$ steps regardless of n . Instances are solved in parallel by OpenMP, one instance per thread, with no interaction between them.

Metric: cumulative CPU time per instance, from the run log — not the `time_ms` column of the CSV. That column is wall time measured *inside* the 24-thread parallel region, so it absorbs memory contention: at $n = 1000$ it overstates the cost by about $2.5\times$ and is not additive, which makes the annealing look roughly

10× cheaper than it is. Use it for latency distributions (third panel), never for cost accounting.

The annealing cost is isolated as $(t(10S) - t(S))/9$ at equal n : both runs pay the same construction, so it cancels exactly. Construction is then $t(S)$ minus that, cross-checked against an independent `--no-sa` run. A 2× budget delta was tried first and is not enough at $n = 1000$, where the run-to-run spread on the construction alone ($\pm 3\%$, measured over three repetitions) exceeds the entire annealing cost.

n	construction	annealing	total	annealing share	--no-sa
20	0.3463	2.8387	3.1850	89.1 %	0.0050
50	0.2399	2.8751	3.1150	92.3 %	0.0190
100	0.4621	2.9259	3.3880	86.4 %	0.0640
200	0.8094	2.9886	3.7980	78.7 %	0.2090
500	2.6179	3.0911	5.7090	54.1 %	1.1460
1000	5.8518	3.1362	8.9880	34.9 %	6.6380

Table 12: CPU time per instance (ms). Empirical scaling on this grid: construction $\sim n^{0.80}$, annealing $\sim n^{0.03}$ at a fixed step count — the step count does not depend on n , so that exponent is pure cache behaviour.

threads	wall (s)	speed-up	efficiency
1	0.560	1.00×	100 %
2	0.291	1.92×	96 %
4	0.158	3.54×	88 %
8	0.093	6.01×	75 %
12	0.072	7.82×	65 %
16	0.060	9.36×	59 %
24	0.064	8.69×	36 %

Table 13: Thread scaling at $n = 100$. Instances are independent, so the loss at high thread counts is memory bandwidth and clock throttling, not synchronisation.

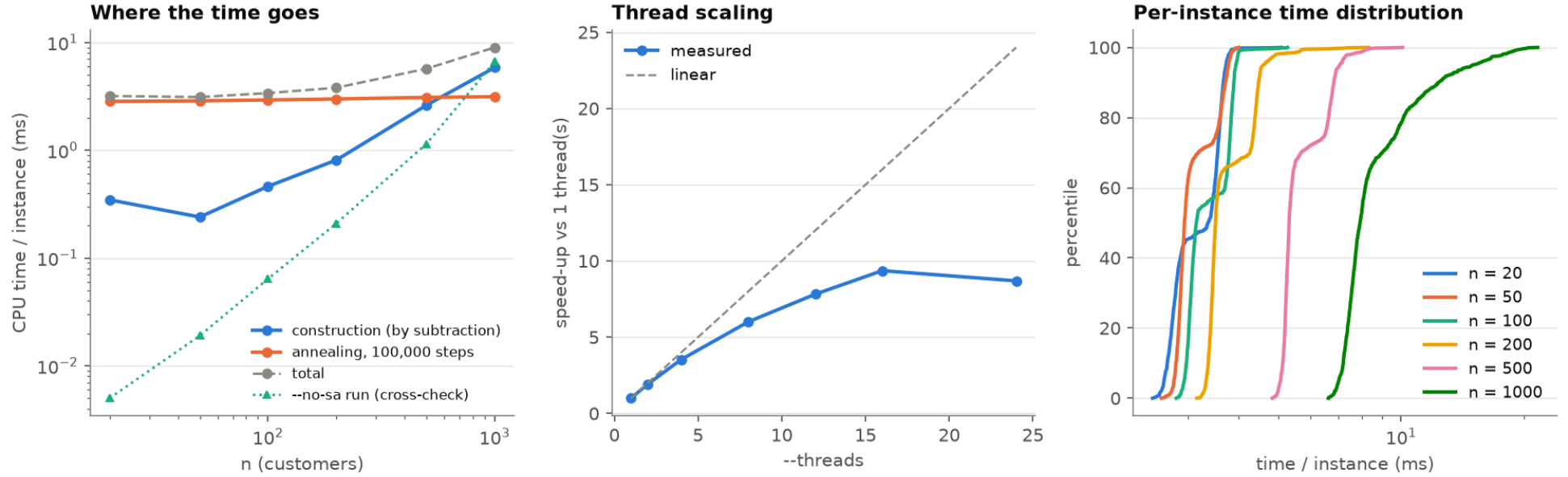


Figure 8: Left: construction and annealing cost against n (log-log). Middle: thread scaling. Right: distribution of per-instance wall time inside the parallel region.

Reading. Up to $n \approx 200$ the annealing is essentially the whole cost and the construction is free; past $n \approx 500$ the $O(n^2)$ savings list takes over and dominates. If large instances ever become the target, the construction is where to look — `--knn` truncates that list, and Section 14 shows the annealing absorbs the loss of quality.

8 Restarts

What it controls. `--restarts R` builds R initial solutions per instance, anneals each one, and keeps the best. Restart 0 is always the deterministic Clarke & Wright; the others are diversified by `--cw-rand: perturb` multiplies every saving by $1 + \alpha U(-1, 1)$ with $\alpha = \text{--cw-alpha}$, which reshuffles the order in which pairs are merged; `param` redraws λ and μ ; `both` does both; `off` makes all restarts identical apart from the annealing seed.

R multiplies the work by R . Comparing R restarts against one at the same `--sa-steps` answers nothing useful — of course more compute helps. The study therefore also holds $R \times \text{sa-steps}$ constant, which asks the question that matters: given a fixed number of annealing steps, is it better to spend them on one long run or several short ones?

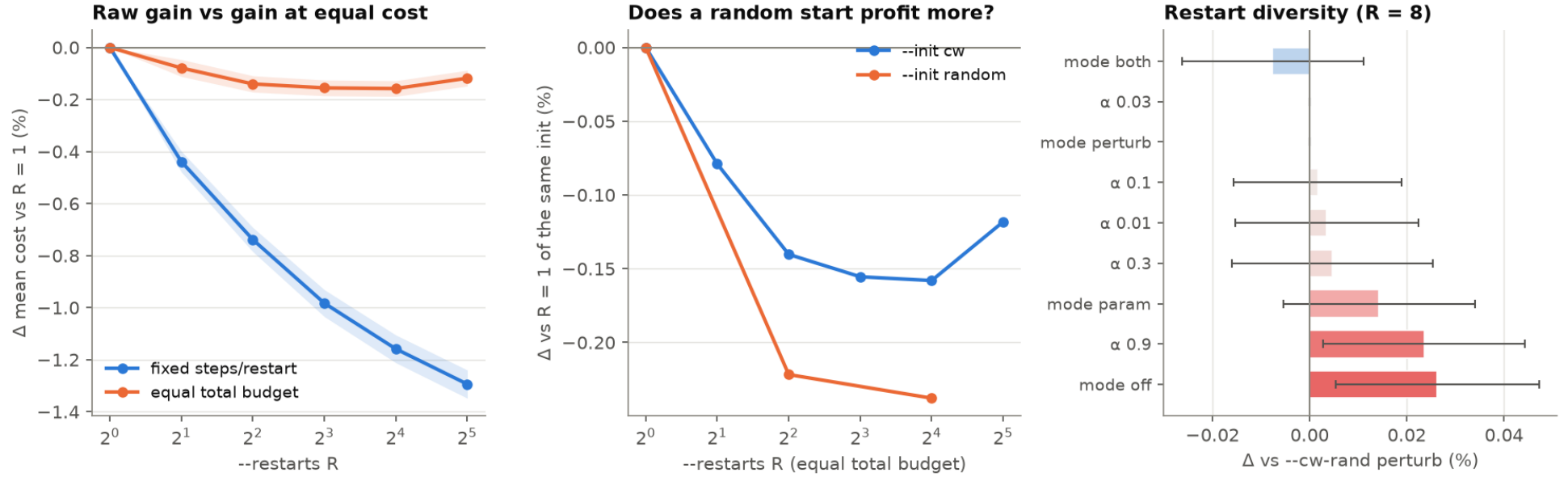


Figure 9: Left: the same restart count judged two ways. Middle: restarts crossed with the initialisation. Right: how the diversity between restarts is generated.

R	steps/restart	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
1	32,000,000	15.46892	+0.000	[+0.000, +0.000]	0/0	15.97
2	16,000,000	15.45670	-0.079	[-0.111, -0.047]	502/434	16.28
4	8,000,000	15.44722	-0.140	[-0.172, -0.109]	552/394	16.10
8	4,000,000	15.44487	-0.155	[-0.186, -0.125]	571/376	16.13
16	2,000,000	15.44447	-0.158	[-0.188, -0.128]	562/389	16.39
32	1,000,000	15.45063	-0.118	[-0.148, -0.088]	529/416	16.28

Table 14: Restarts at *equal total budget* ($R \times \text{sa-steps} = 320\,000$), paired against $R = 1$.

R	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
1	15.65317	+0.000	[+0.000, +0.000]	0/0	0.53
2	15.58450	-0.439	[-0.478, -0.399]	516/0	1.04
4	15.53759	-0.738	[-0.785, -0.692]	769/0	2.06
8	15.49935	-0.983	[-1.034, -0.931]	886/0	4.10
16	15.47185	-1.158	[-1.212, -1.105]	939/0	8.21
32	15.45063	-1.294	[-1.348, -1.240]	965/0	16.41

Table 15: Restarts at fixed steps per restart — i.e. at R times the compute. Shown for completeness; the table above is the meaningful one.

variant	mean cost	Δ (%)	95 % CI	win/loss
mode both	15.44369	-0.008	[-0.026, +0.011]	428/422
alpha 0.03	15.44487	+0.000	[+0.000, +0.000]	0/0
mode perturb	15.44487	+0.000	[+0.000, +0.000]	0/0
alpha 0.1	15.44513	+0.002	[-0.016, +0.019]	429/428
alpha 0.01	15.44541	+0.004	[-0.015, +0.022]	421/450
alpha 0.3	15.44559	+0.005	[-0.016, +0.025]	434/426
mode param	15.44708	+0.014	[-0.005, +0.034]	426/433
alpha 0.9	15.44851	+0.024	[+0.003, +0.044]	399/472
mode off	15.44893	+0.026	[+0.005, +0.047]	425/437

Table 16: How restart diversity is produced, at $R = 8$, against the default `perturb` with $\alpha = 0.03$.

Reading. At equal budget, splitting the run pays: 16 restarts of 2 000 000 steps each beat a single run of 32 000 000 by -0.158% , and 5 of the 5 restart counts tried clear significance. The annealing *is* getting stuck in a way a fresh start fixes — so at this budget the whole allowance should not go into one long anneal. The random start is the exception — it degrades sharply as its share of the budget shrinks, which is the same result as Section 3 seen from another angle. On diversity, the default `perturb` at $\alpha = 0.03$ is the best of the options: too much perturbation ($\alpha \geq 0.3$) damages every restart, and `off` is worse still.

9 Optimal Split

What it controls. Split (Vidal, 2016) takes the current solution, concatenates its routes into one giant tour ignoring capacity, then re-cuts that tour into routes optimally by dynamic programming in $O(n)$. Because the current partition is one of the candidates, the result can never be worse. `--split` says where to apply it: `cw` right after the construction, `end` after the annealing, `both`, or `off`. `--split-every N` additionally applies it every N annealing steps — an independent code path (`cw.c:1744` versus `cw.c:1968/2075`), so `off` combined with a period is a meaningful cell. `--split-tour` chooses how the giant tour is built: routes in their current order, `sweep` by polar angle, or `both` keeping the better.

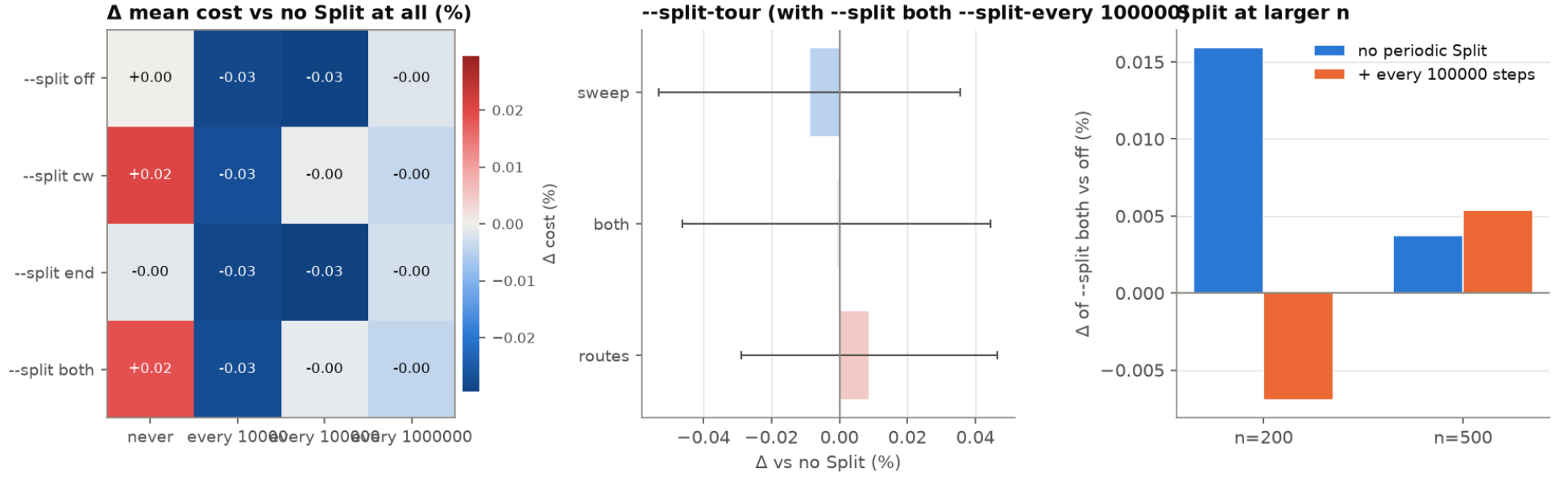


Figure 10: Left: the full mode $\times$ period grid. Middle: giant-tour construction. Right: the same comparison at larger n , where there are more routes to repartition.

Table 17: Split mode against periodic Split, paired against no Split at all. ‘Split gain’ is the total improvement cw attributes to Split internally — note it is large where the paired delta is not, i.e. Split keeps recovering cost the annealing had just spent.

--split	--split-every	mean cost	Δ (%)	95 % CI	win/loss	Split gain	wall (s)
off	never	15.50934	+0.000	[+0.000, +0.000]	0/0	—	5.08
off	10000	15.50497	-0.028	[-0.072, +0.015]	509/468	4.1218	5.15
off	100000	15.50483	-0.029	[-0.070, +0.012]	492/467	0.3894	5.08
off	1000000	15.50909	-0.002	[-0.035, +0.032]	495/459	0.0231	5.15
cw	never	15.51251	+0.020	[-0.023, +0.063]	489/475	0.0000	5.06
cw	10000	15.50516	-0.027	[-0.070, +0.016]	528/451	4.1115	5.14
cw	100000	15.50927	-0.000	[-0.046, +0.045]	509/469	0.3976	5.12
cw	1000000	15.50875	-0.004	[-0.047, +0.040]	499/471	0.0230	5.11
end	never	15.50916	-0.001	[-0.002, -0.001]	34/0	0.0002	5.07
end	10000	15.50492	-0.029	[-0.072, +0.015]	509/468	4.1219	5.16
end	100000	15.50477	-0.030	[-0.071, +0.012]	492/467	0.3895	5.12
end	1000000	15.50895	-0.003	[-0.036, +0.031]	496/458	0.0233	5.10

--split	--split-every	mean cost	Δ (%)	95 % CI	win/loss	Split gain	wall (s)
both	never	15.51227	+0.019	[-0.024, +0.062]	489/475	0.0002	5.04
both	10000	15.50512	-0.027	[-0.070, +0.016]	528/451	4.1115	5.21
both	100000	15.50921	-0.001	[-0.046, +0.044]	509/469	0.3977	5.15
both	1000000	15.50867	-0.004	[-0.048, +0.039]	500/470	0.0231	5.15

Reading. Split never hurts: `--split end` — one Split, after the annealing — improves 34 instances and worsens 0, which is the theoretical guarantee showing up in the data. But on this instance family it barely helps either: the best cell in the whole grid is `end` + every 100,000 at -0.030% . Its internal ‘gain’ counter is large while the net effect is small, meaning it mostly undoes damage the annealing did rather than finding new structure. Applying it every 10,000 steps — the most aggressive period tried — costs $1.0\times$ the wall time (5.16s against 5.08s) and is worth at worst -0.027% : it drags the search back to a repartitioned solution before the annealing can exploit its own moves.

10 Vertex selection

What it controls. Before a move can be built, the annealing must pick the vertex u to disturb (`pick_u`, `cw.c:987`). Uniform choice wastes draws on customers that are already well placed, so the code biases the choice by a *regret* measure. With `--pick  $T \geq 2$` it draws T customers uniformly and keeps the one with the largest regret (a tournament); $T = 1$ is plain uniform; $T = 0$ samples exactly proportional to $\max(\text{regret}, 0) + \varepsilon$ through a Fenwick tree, at $O(\log n)$ per draw and per update.

`--pick-crit` defines the regret, with $\text{inc}[u] = d(\text{prv}[u], u) + d(u, \text{nxt}[u])$ the cost of the two edges u currently carries:

- `lb` (default) — $\text{inc}[u] - \text{lb}[u]$, the gap to the two shortest edges available to u in its kNN list.
- `rem` — $\text{inc}[u] - d(p, q)$, the removal gain: exactly what relocating u could recover at best.
- `remnorm` — the removal gain divided by $\text{lb}[u]$, normalising by the local density.
- `raw` — $\text{inc}[u]$ itself.

`--pick-eps` sets how peaked the $T = 0$ sampler is.

The code’s own comment (`cw.c:938-916`) notes that `lb` “can stay large for an isolated customer even when nothing can improve it any further”, whereas `rem` is zero as soon as u sits well. That suggests the default wastes draws on structurally irreducible vertices. The grid below was run specifically to test that prediction.

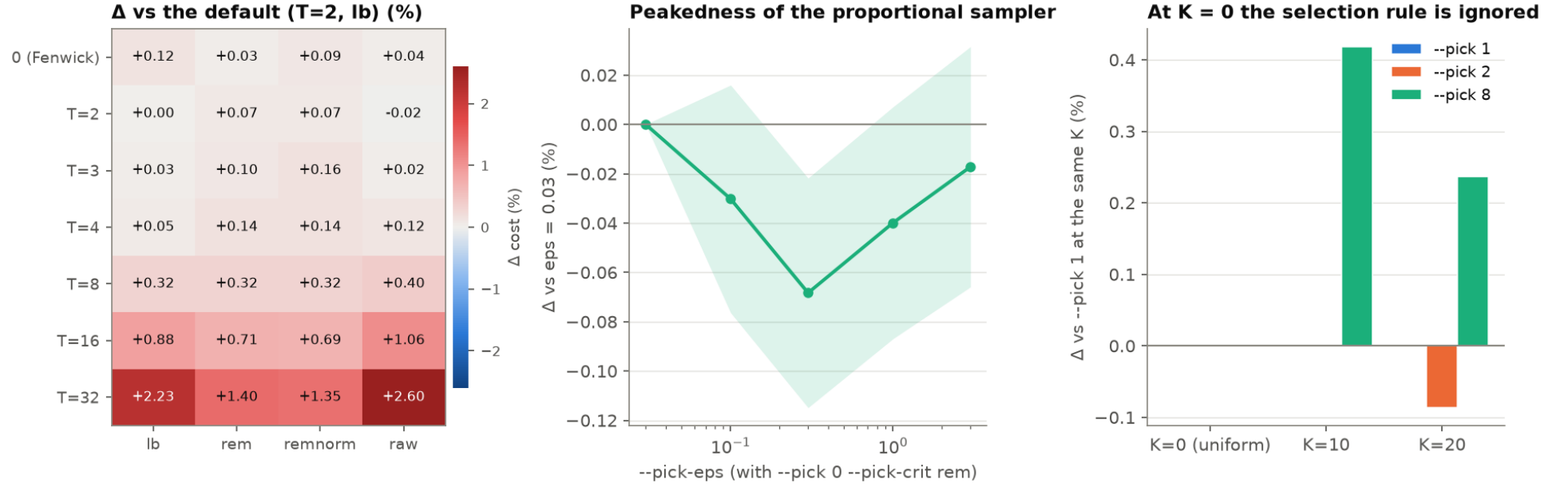


Figure 11: Left: tournament size against regret criterion. Middle: peakedness of the proportional sampler. Right: `--pick` crossed with `--sa-knn`; the flat $K = 0$ group is the silent fallback.

Table 18: Tournament size against regret criterion, paired against the default $T = 2$ with `lb`.

--pick	--pick-crit	mean cost	Δ (%)	95 % CI	win/loss
1 (uniform)	–	15.52275	+0.086	[+0.040, +0.132]	444/532
0 (Fenwick)	lb	15.52841	+0.123	[+0.077, +0.168]	420/552
0 (Fenwick)	rem	15.51350	+0.027	[-0.019, +0.072]	489/487
0 (Fenwick)	remnorm	15.52366	+0.092	[+0.043, +0.141]	456/529
0 (Fenwick)	raw	15.51483	+0.035	[-0.008, +0.079]	467/506
2	lb	15.50934	+0.000	[+0.000, +0.000]	0/0
2	rem	15.52041	+0.071	[+0.027, +0.116]	457/528
2	remnorm	15.52076	+0.074	[+0.030, +0.117]	472/511
2	raw	15.50644	-0.019	[-0.062, +0.024]	496/473
3	lb	15.51347	+0.027	[-0.016, +0.069]	472/511
3	rem	15.52428	+0.096	[+0.050, +0.143]	447/532
3	remnorm	15.53412	+0.160	[+0.112, +0.207]	413/576
3	raw	15.51308	+0.024	[-0.019, +0.067]	474/508

--pick	--pick-crit	mean cost	Δ (%)	95 % CI	win/loss
4	lb	15.51711	+0.050	[+0.005, +0.095]	457/519
4	rem	15.53167	+0.144	[+0.095, +0.193]	432/563
4	remnorm	15.53180	+0.145	[+0.098, +0.192]	411/568
4	raw	15.52734	+0.116	[+0.071, +0.161]	423/560
8	lb	15.55948	+0.323	[+0.278, +0.369]	306/685
8	rem	15.55869	+0.318	[+0.268, +0.368]	327/663
8	remnorm	15.55834	+0.316	[+0.264, +0.368]	345/650
8	raw	15.57212	+0.405	[+0.357, +0.452]	284/707
16	lb	15.64659	+0.885	[+0.825, +0.945]	141/855
16	rem	15.61894	+0.707	[+0.648, +0.766]	220/774
16	remnorm	15.61644	+0.690	[+0.631, +0.750]	226/774
16	raw	15.67407	+1.062	[+1.005, +1.119]	102/896
32	lb	15.85464	+2.226	[+2.146, +2.306]	24/976
32	rem	15.72615	+1.398	[+1.333, +1.463]	77/923
32	remnorm	15.71869	+1.350	[+1.284, +1.416]	81/919
32	raw	15.91279	+2.601	[+2.520, +2.683]	8/992

--sa-knn	--pick	mean cost	Δ (%)	95 % CI	win/loss
0	1	15.62428	+0.000	[+0.000, +0.000]	0/0
10	1	15.56186	+0.000	[+0.000, +0.000]	0/0
20	1	15.52275	+0.000	[+0.000, +0.000]	0/0
0	2	15.62428	+0.000	[+0.000, +0.000]	0/0
10	2	15.56204	+0.001	[-0.046, +0.049]	478/507
20	2	15.50934	-0.086	[-0.132, -0.040]	532/444
0	8	15.62428	+0.000	[+0.000, +0.000]	0/0
10	8	15.62694	+0.418	[+0.363, +0.474]	310/685
20	8	15.55948	+0.237	[+0.187, +0.286]	369/623

Table 19: --pick crossed with --sa-knn, paired against --pick 1 at the same K . The $K = 0$ rows are exactly zero because `cw.c:1676` forces uniform selection there — while the run header still prints the requested rule.

Reading. At $n = 100$ with 10 000 000 steps the entire grid — every tournament size, every criterion, and the Fenwick sampler — sits inside $\pm 2.601\%$ of the default, and 22 of the 29 cells clears significance. The regret machinery is measurable in the code but not, at this budget, in the result: whatever it saves in draw quality it appears to give back in draw cost. The worst cell is $T = 32$ with **raw** at $+2.601\%$, from over-concentrating on a few vertices; the best is $T = 2$ with **raw** at -0.019% . The one unambiguous finding here is the silent fallback at $K = 0$: the header reports a rule the code is not using.

11 Biasing the second vertex

What it controls. Section 10 is about choosing the vertex u to move. These three knobs are about the *partner* v , and about where u is put once chosen.

- `--vrank T` — the kNN lists are stored sorted by distance, so drawing the index as the *minimum of T uniform draws* biases towards the nearer neighbours at no memory access at all. $T = 1$ is the plain uniform draw.
- `--pick2 T` — a tournament of size T among candidate partners, keeping the one of largest regret. Unlike `--vrank` this one costs a memory read per candidate.
- `--reloc-side long` — relocate breaks the *longer* of the two edges adjacent to v instead of flipping a coin, which maximises the $-d(v, q)$ term of the insertion cost. Two reads, no randomness.

The question. `--vrank` is claimed to be *coupled* with `--sa-knn`: a longer candidate list gives the reach, the rank bias restores the concentration, and either one alone is supposed to hurt. That is only testable on the grid, so the grid is what is run.

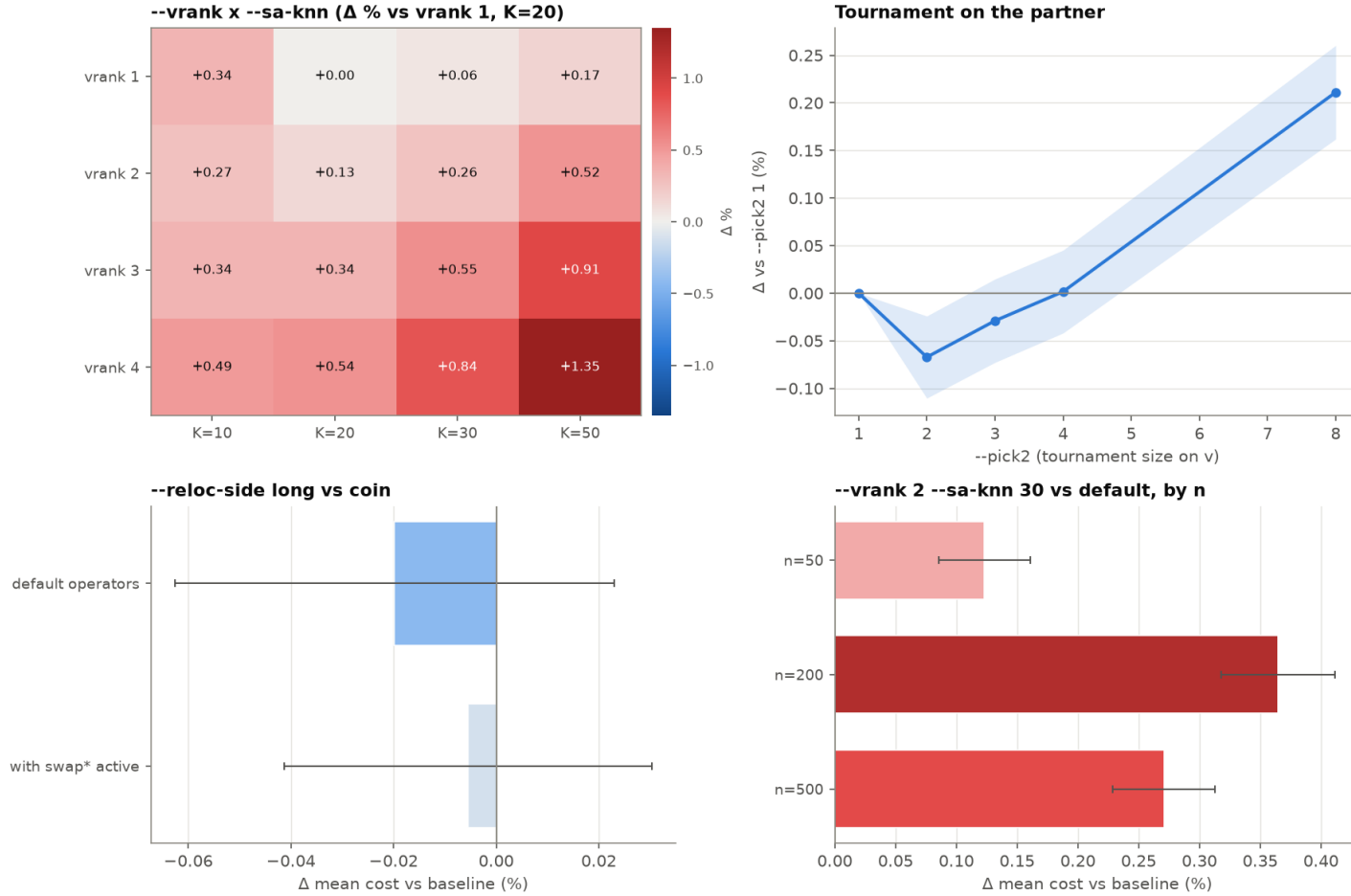


Figure 12: Top left: the `--vrnk`/`--sa-knn` grid, the coupling claim. Top right: tournament size on the partner. Bottom left: the relocate insertion side. Bottom right: the recommended `--vrnk 2 --sa-knn 30` pair across dimension.

Table 20: The full `--vrnk` $\times$ `--sa-knn` grid, paired against `vrnk 1`, K=20 (the default).

--vrnk	--sa-knn	mean cost	Δ (%)	95 % CI	win/loss
1	10	15.56204	+0.340	[+0.288, +0.391]	329/657

--vrank	--sa-knn	mean cost	Δ (%)	95 % CI	win/loss
1	20	15.50934	+0.000	[+0.000, +0.000]	0/0
1	30	15.51840	+0.058	[+0.012, +0.105]	456/526
1	50	15.53535	+0.168	[+0.118, +0.217]	405/574
2	10	15.55060	+0.266	[+0.218, +0.314]	353/636
2	20	15.52976	+0.132	[+0.085, +0.178]	412/570
2	30	15.54966	+0.260	[+0.210, +0.310]	352/637
2	50	15.58971	+0.518	[+0.465, +0.571]	250/747
3	10	15.56212	+0.340	[+0.291, +0.390]	312/674
3	20	15.56275	+0.344	[+0.294, +0.395]	310/680
3	30	15.59409	+0.546	[+0.493, +0.600]	247/752
3	50	15.65099	+0.913	[+0.852, +0.974]	147/853
4	10	15.58534	+0.490	[+0.439, +0.541]	263/732
4	20	15.59283	+0.538	[+0.486, +0.590]	239/759
4	30	15.63944	+0.839	[+0.780, +0.897]	179/819
4	50	15.71894	+1.351	[+1.286, +1.417]	78/920

context	coin	long	Δ (%)	95 % CI	win/loss
default operators	15.50934	15.50627	-0.020	[-0.063, +0.023]	504/467
with swap* active	15.45920	15.45834	-0.006	[-0.041, +0.030]	478/457

Table 21: The relocate insertion side, with and without swap* active.

n	default	vrank 2, K=30	Δ (%)	95 % CI	win/loss
50	10.27040	10.28304	+0.123	[+0.085, +0.161]	268/480
200	21.96580	22.04590	+0.365	[+0.318, +0.411]	321/679
500	37.14129	37.24189	+0.271	[+0.229, +0.313]	349/651

Table 22: The rank bias with the longer candidate list, across dimension.

Reading. The coupling claim does not reproduce here, and the heat map is unusually unambiguous about it: the best cell of the whole grid is the default corner **vrank 1**, **K=20**, and the supposed winning combination **vrank 2**, **K=30** ranks 5 of 16 cells tried. The most likely explanation for the disagreement with the source is that the claim was made against a construction that shares one $K = 30$ neighbour list between the savings and the annealing, whereas here the savings are exact for $n \leq 1500$ and the annealing builds its own list. Those are not the same baseline, and this sweep cannot settle which is right for the other one — only what happens on this code at this budget.

--pick2 is the mild surprise: a tournament of 3 on the partner is a small but genuine improvement, which makes it the only selection bias in this document that survives while costing a memory read per candidate. It is worth about a tenth of what swap* is worth, and the curve is flat enough between 2 and 4 that the exact size does

not matter. The relocate insertion side moves in the right direction with and without `swap*` active, but neither interval excludes zero: on this evidence it is free rather than useful, and it is left off.

12 Racing between restarts, and interleaving

What it controls. Section 8 spends the budget equally over the restarts. `--race` M spends it unequally instead: at `--race-at` of its own budget (a quarter by default) a start is compared with the best *finished* start *at the same point of its trajectory* — same temperature, which is what makes the comparison fair — and abandoned if it is worse by a relative margin M . Its unspent steps lengthen the schedules of the starts that follow, and whatever is left at the end is spent polishing the best solution found. The total step budget never changes; only its allocation does.

`--pair` is a different kind of knob: two starts are advanced alternately in the same loop so that one’s memory latency overlaps with the other’s arithmetic. Each chain owns its solution, its incremental buffers and its random stream, so without `--race` the trajectories are identical bit for bit and only the clock may move. That makes it a self-checking row in the table below: if the cost column is not flat, something is wrong.

The question. Every run in this section holds `restarts` $\times$ `sa-steps` constant, because a racing gain at unequal budget would be meaningless.

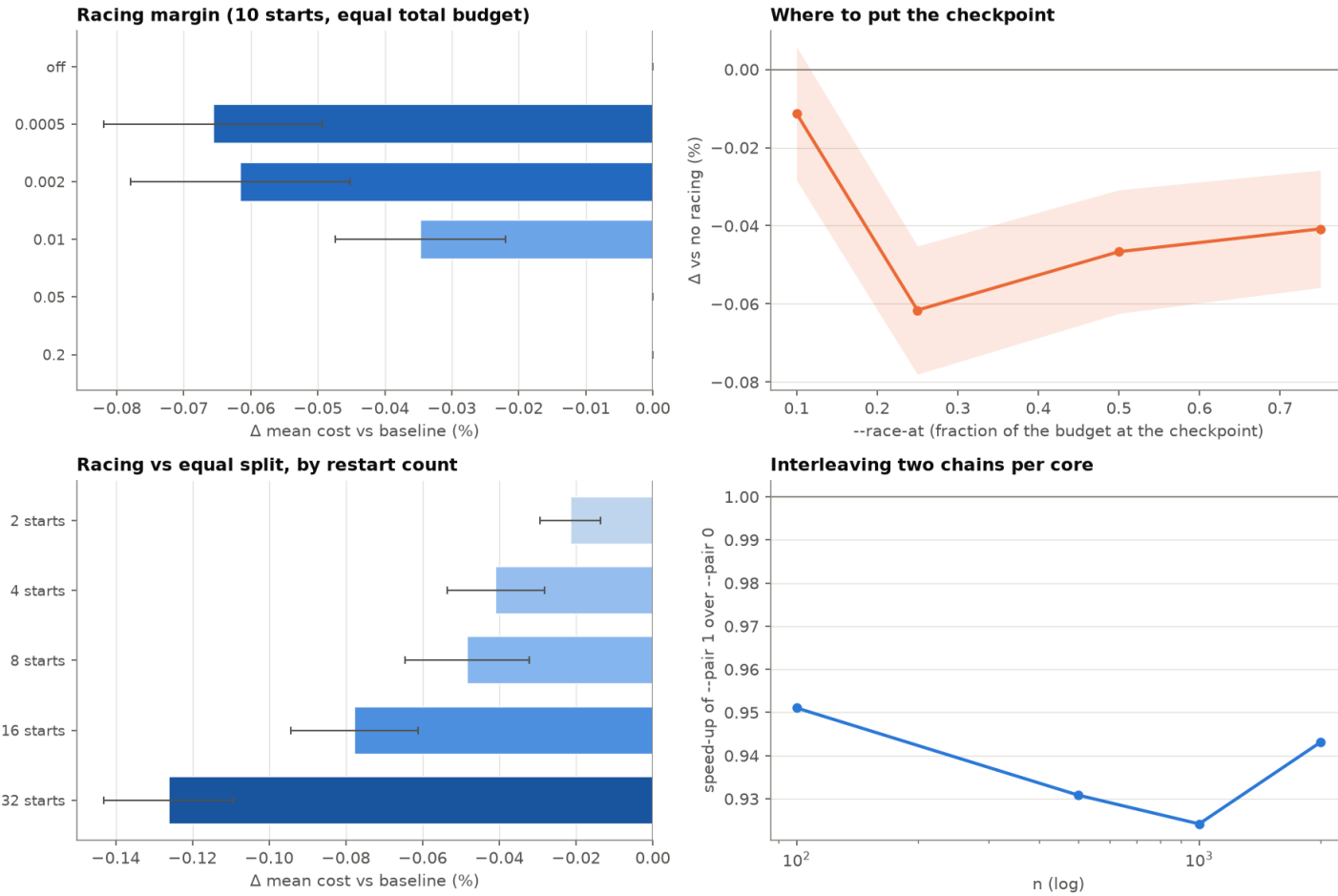


Figure 13: Top: the racing margin and the checkpoint position, both at equal total budget. Bottom left: racing against an equal split as the number of starts grows. Bottom right: the interleaving speed-up, which is a timing result only — the costs are identical.

margin	mean cost	Δ (%)	95 % CI	win/loss
off	15.43882	+0.000	[+0.000, +0.000]	0/0
0.0005	15.42869	-0.066	[-0.082, -0.049]	494/293
0.002	15.42931	-0.062	[-0.078, -0.045]	477/281
0.01	15.43346	-0.035	[-0.047, -0.022]	320/183
0.05	15.43882	+0.000	[+0.000, +0.000]	0/0
0.2	15.43882	+0.000	[+0.000, +0.000]	0/0

Table 23: Racing margin, 10 starts at equal total budget.

--race-at	mean cost	Δ (%)	95 % CI	win/loss
0.1	15.43708	-0.011	[-0.028, +0.006]	397/365
0.25	15.42931	-0.062	[-0.078, -0.045]	477/281
0.5	15.43162	-0.047	[-0.062, -0.031]	464/294
0.75	15.43252	-0.041	[-0.056, -0.026]	429/311

Table 24: Position of the checkpoint, at margin 0.002.

starts	equal split	racing	Δ (%)	95 % CI	win/loss
2	15.44519	15.44187	-0.021	[-0.029, -0.014]	88/23
4	15.44427	15.43794	-0.041	[-0.054, -0.028]	283/159
8	15.43895	15.43148	-0.048	[-0.065, -0.032]	417/291
16	15.43873	15.42673	-0.078	[-0.094, -0.061]	511/310
32	15.44386	15.42437	-0.126	[-0.143, -0.109]	633/250

Table 25: Racing against an equal split, holding `restarts` $\times$ `sa-steps` constant.

n	--pair 0	--pair 1	identical	wall 0	wall 1	speed-up
100	15.43895	15.43895	yes	20.08s	21.11s	0.95x
500	37.11455	37.11455	yes	21.72s	23.33s	0.93x
1000	43.00602	43.00602	yes	23.57s	25.50s	0.92x
2000	77.35325	77.35325	yes	24.77s	26.26s	0.94x

Table 26: Interleaving. The *identical* column is the check that this is an engineering change and not an algorithmic one: the two costs must agree exactly.

Reading. Racing needs starts to race: with 2 starts it is worth -0.021% and with 32 -0.126% , so the gain grows with the restart count. It costs nothing — the budget is the same, only its allocation changes.

The margin behaves monotonically over the range tried, which is worth noting because it was not the expectation: the tightest margin is the best one, and loosening it only ever gives ground back. By 0.05 almost no start is ever behind by that much at the checkpoint, so the setting is indistinguishable from `off`, and by 0.2 it is exactly `off`. There was reason to expect the other failure mode as well — a margin so tight that good starts are killed on noise — and this sweep does not reach it, so the bottom of the useful range is still unmeasured. The checkpoint position matters much less than the margin; anything from a tenth to half of the budget is within noise of the default quarter, and only pushing it out to three quarters clearly hurts, by which point most of the budget the racing was meant to reclaim has already been spent.

`--pair` does exactly what it claims. The *identical* column reads `yes` on every row — the interleaved chains reproduce the sequential trajectories exactly — and the speed-up appears only once the state of one chain stops fitting in the low-level caches: slightly negative at $n \leq 500$, where the bookkeeping is not paid back, and clearly positive from $n = 1000$. That is a smaller effect than the $1.42\times$ reported for the implementation this was ported from; the shape of the curve is the same, so the difference is most likely the cache hierarchy of the machine rather than the technique.

13 Annealing schedule

What it controls. The initial temperature T_0 is not given directly: `calibrate_T0` samples worsening moves on the actual instance and solves for the T_0 whose acceptance probability matches `--t-accept` (default 0.001, i.e. 0.1 % of worsening moves accepted at the start). The temperature then decays geometrically to $T_0 \cdot 10^{-D}$ over the whole run, with $D = \text{--t-decades}$ (default 2). Together they fix both ends of the schedule; `--t0` and `--tend` override the calibration.

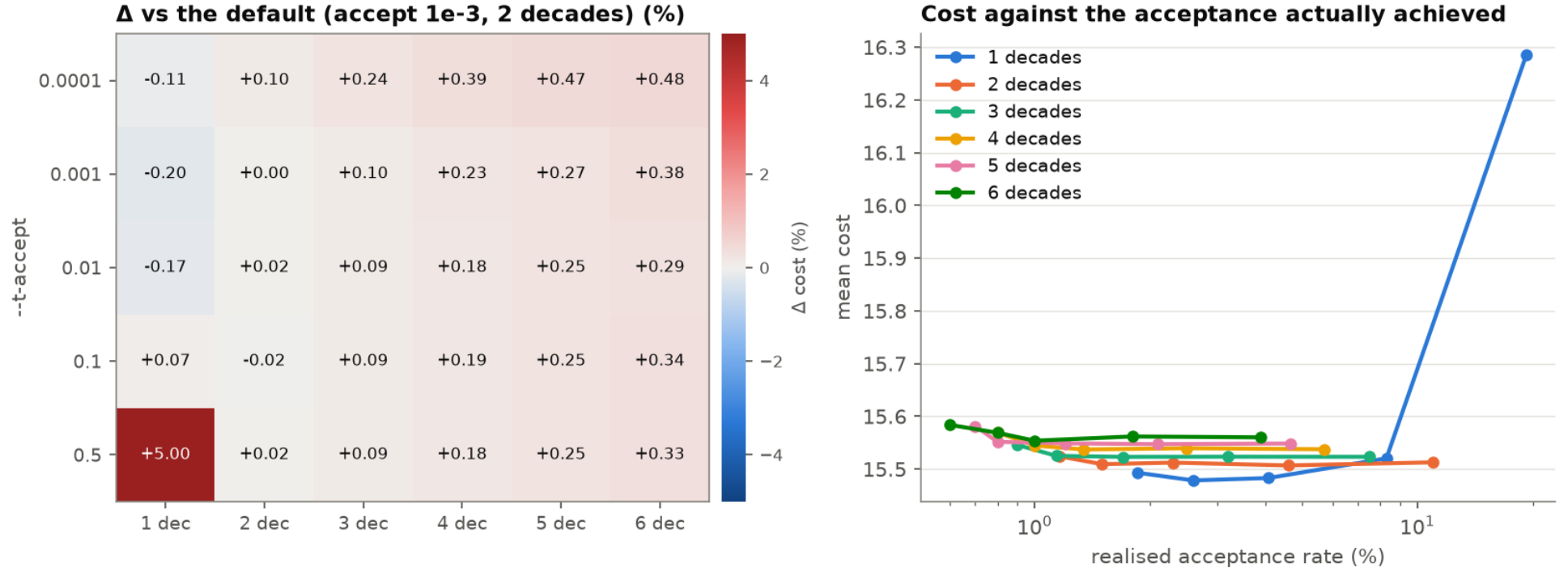


Figure 14: Left: the full `--t-accept` $\times$ `--t-decades` grid. Right: the same runs plotted against the acceptance rate they actually realised.

Table 27: Annealing schedule, paired against the default (`t-accept` = 10^{-3} , 2 decades).

--t-accept	--t-decades	mean cost	accept. (%)	Δ (%)	95 % CI	win/loss
0.0001	1	15.49302	1.86	-0.105	[-0.147, -0.064]	568/395
0.0001	2	15.52430	1.16	+0.096	[+0.051, +0.142]	441/538
0.0001	3	15.54587	0.90	+0.236	[+0.188, +0.283]	387/594
0.0001	4	15.56960	0.80	+0.389	[+0.337, +0.440]	322/662
0.0001	5	15.58159	0.70	+0.466	[+0.412, +0.519]	298/691
0.0001	6	15.58394	0.60	+0.481	[+0.429, +0.533]	276/710
0.001	1	15.47852	2.60	-0.199	[-0.240, -0.158]	607/356
0.001	2	15.50934	1.50	+0.000	[+0.000, +0.000]	0/0
0.001	3	15.52514	1.14	+0.102	[+0.058, +0.146]	433/536
0.001	4	15.54482	1.00	+0.229	[+0.183, +0.275]	384/594

--t-accept	--t-decades	mean cost	accept. (%)	Δ (%)	95 % CI	win/loss
0.001	5	15.55167	0.80	+0.273	[+0.223, +0.323]	359/630
0.001	6	15.56887	0.80	+0.384	[+0.332, +0.436]	323/664
0.01	1	15.48306	4.08	-0.169	[-0.212, -0.127]	588/388
0.01	2	15.51225	2.30	+0.019	[-0.025, +0.062]	500/474
0.01	3	15.52318	1.70	+0.089	[+0.045, +0.133]	434/547
0.01	4	15.53724	1.34	+0.180	[+0.133, +0.227]	383/598
0.01	5	15.54877	1.20	+0.254	[+0.205, +0.303]	363/622
0.01	6	15.55381	1.00	+0.287	[+0.239, +0.335]	352/636
0.1	1	15.52033	8.30	+0.071	[+0.026, +0.115]	409/591
0.1	2	15.50697	4.60	-0.015	[-0.060, +0.029]	498/473
0.1	3	15.52348	3.20	+0.091	[+0.047, +0.135]	431/547
0.1	4	15.53956	2.50	+0.195	[+0.148, +0.241]	384/596
0.1	5	15.54743	2.10	+0.246	[+0.196, +0.295]	360/624
0.1	6	15.56185	1.80	+0.339	[+0.289, +0.388]	315/674
0.5	1	16.28548	19.26	+5.004	[+4.911, +5.097]	0/1000
0.5	2	15.51278	10.98	+0.022	[-0.023, +0.067]	494/483
0.5	3	15.52328	7.50	+0.090	[+0.043, +0.137]	437/539
0.5	4	15.53763	5.70	+0.182	[+0.136, +0.229]	380/608
0.5	5	15.54854	4.66	+0.253	[+0.205, +0.301]	345/635
0.5	6	15.56013	3.90	+0.327	[+0.275, +0.380]	316/670

Reading. The best cell at this budget is `--t-accept` = 0.001 with 1 decade, worth -0.199% against the default, and it realises a 2.60% acceptance rate. The mechanism is that a schedule spanning too many decades spends the tail of the run at temperatures so low that nothing is accepted — the search is effectively over before the step budget is — while too few leaves it still hot at the end. Where that trade-off lands is a function of the budget: the geometric ratio is $(T_{\text{end}}/T_0)^{1/(S-1)}$, so at $S = 10^7$ the same decade count is traversed a hundred times more slowly than at 10^5 . Section 2 reports whether the two budgets agree here; if they do not, this is the knob to re-measure whenever `--sa-steps` changes substantially.

14 Construction quality

What it controls. Three separate things about the Clarke & Wright stage. The savings formula $s(i, j) = d_{0i} + d_{0j} - \lambda d_{ij} + \mu |d_{0i} - d_{0j}|$ has two shape parameters: `--lambda` sets how much weight the direct link between two customers carries against their two depot legs (the classical generalisation of Clarke & Wright, $\lambda > 1$ favouring longer, less radial routes), and `--mu` adds an asymmetry term penalising the merge of two customers at very different distances from the depot. `--knn` K truncates the savings list to each customer’s K nearest neighbours instead of all $O(n^2)$ pairs (`--exact` forces the full list; the default is exact for $n \leq 1500$). `--2opt` runs an intra-route 2-opt pass after the construction.

The question. Every one of these improves or degrades the *starting* solution. The interesting question is whether that survives the annealing, so each grid is run twice: with `--no-sa` and with the full budget.

Table 28: Savings parameters, construction only, paired against $\lambda = 1$, $\mu = 0$.

λ	μ	mean cost	Δ (%)	95 % CI	win/loss
0.6	0	16.91801	+3.695	[+3.541, +3.850]	50/950
0.6	0.2	16.91901	+3.701	[+3.565, +3.838]	53/947
0.6	0.5	16.94591	+3.866	[+3.720, +4.013]	48/952
0.6	1	16.98345	+4.096	[+3.935, +4.258]	63/937
0.8	0	16.51797	+1.243	[+1.137, +1.350]	199/801
0.8	0.2	16.56313	+1.520	[+1.407, +1.633]	168/832
0.8	0.5	16.66852	+2.166	[+2.042, +2.290]	149/851
0.8	1	16.87840	+3.453	[+3.304, +3.602]	83/917
1	0	16.31511	+0.000	[+0.000, +0.000]	0/0
1	0.2	16.36134	+0.283	[+0.190, +0.377]	404/596
1	0.5	16.49622	+1.110	[+0.983, +1.237]	304/696
1	1	16.75308	+2.684	[+2.545, +2.823]	123/877
1.2	0	16.29797	-0.105	[-0.204, -0.006]	552/448
1.2	0.2	16.23307	-0.503	[-0.599, -0.406]	601/374
1.2	0.5	16.34296	+0.171	[+0.057, +0.285]	464/536
1.2	1	16.62678	+1.910	[+1.778, +2.042]	188/812
1.4	0	16.36424	+0.301	[+0.184, +0.418]	462/538
1.4	0.2	16.24143	-0.452	[-0.566, -0.337]	629/371
1.4	0.5	16.25238	-0.384	[-0.498, -0.271]	562/438
1.4	1	16.53563	+1.352	[+1.220, +1.484]	271/729

Table 29: Savings parameters, after annealing, paired against $\lambda = 1$, $\mu = 0$.

λ	μ	mean cost	Δ (%)	95 % CI	win/loss
0.6	0	15.51634	+0.045	[-0.007, +0.097]	482/495
0.6	0.2	15.51216	+0.018	[-0.031, +0.067]	490/479
0.6	0.5	15.51528	+0.038	[-0.010, +0.087]	454/521
0.6	1	15.51370	+0.028	[-0.017, +0.073]	463/500
0.8	0	15.50863	-0.005	[-0.047, +0.038]	496/479
0.8	0.2	15.51360	+0.027	[-0.018, +0.072]	479/496
0.8	0.5	15.50711	-0.014	[-0.058, +0.029]	492/479
0.8	1	15.51678	+0.048	[-0.001, +0.096]	487/478
1	0	15.50934	+0.000	[+0.000, +0.000]	0/0
1	0.2	15.50634	-0.019	[-0.063, +0.024]	506/473
1	0.5	15.51209	+0.018	[-0.026, +0.062]	486/490
1	1	15.50718	-0.014	[-0.059, +0.031]	504/471

λ	μ	mean cost	Δ (%)	95 % CI	win/loss
1.2	0	15.50774	-0.010	[-0.054, +0.033]	509/459
1.2	0.2	15.50276	-0.042	[-0.085, +0.000]	504/440
1.2	0.5	15.50998	+0.004	[-0.040, +0.048]	490/482
1.2	1	15.51185	+0.016	[-0.028, +0.061]	485/493
1.4	0	15.50810	-0.008	[-0.052, +0.036]	489/492
1.4	0.2	15.50867	-0.004	[-0.049, +0.040]	497/478
1.4	0.5	15.51668	+0.047	[+0.002, +0.093]	474/502
1.4	1	15.51009	+0.005	[-0.038, +0.048]	475/494

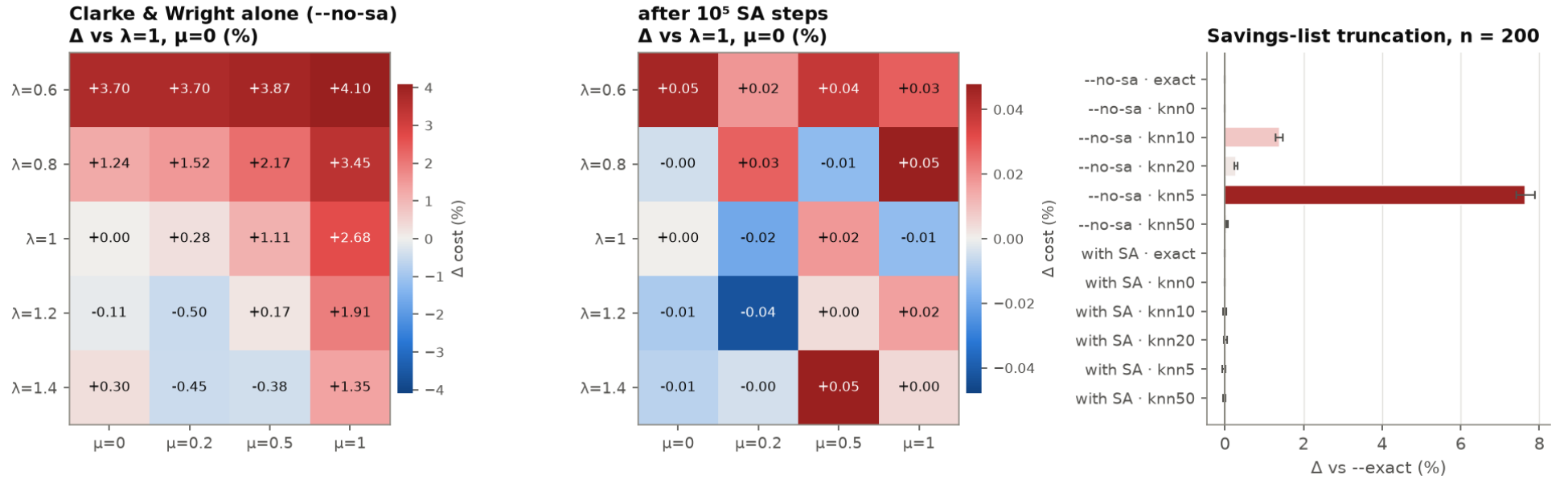


Figure 15: Left and middle: the same $\lambda \times \mu$ grid before and after annealing — note the colour scales differ by an order of magnitude. Right: truncating the savings list, with and without annealing.

stage	variant	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
--no-sa	exact	23.21002	+0.000	[+0.000, +0.000]	0/0	0.01
--no-sa	knn0	23.21002	+0.000	[+0.000, +0.000]	0/0	0.01
--no-sa	knn10	23.53203	+1.387	[+1.292, +1.483]	130/870	0.01
--no-sa	knn20	23.27715	+0.289	[+0.250, +0.329]	215/736	0.01
--no-sa	knn5	24.98497	+7.647	[+7.413, +7.882]	5/995	0.00
--no-sa	knn50	23.22354	+0.058	[+0.044, +0.073]	99/429	0.02
with SA	exact	21.96580	+0.000	[+0.000, +0.000]	0/0	5.21
with SA	knn0	21.96580	+0.000	[+0.000, +0.000]	0/0	5.18
with SA	knn10	21.96487	-0.004	[-0.049, +0.040]	497/503	5.21
with SA	knn20	21.96810	+0.010	[-0.032, +0.053]	471/480	5.19
with SA	knn5	21.96172	-0.019	[-0.062, +0.025]	516/484	5.27
with SA	knn50	21.96284	-0.013	[-0.049, +0.022]	262/266	5.25

Table 30: Savings-list truncation at $n = 200$, paired against `--exact`. `knn0` is the automatic setting, which is exact at this size.

stage	without --2opt	with --2opt	Δ (%)	95 % CI	win/loss
construction only	16.31511	16.24180	-0.449	[-0.469, -0.429]	945/0
after annealing	15.50934	15.51797	+0.056	[+0.013, +0.098]	422/500

Table 31: Intra-route 2-opt after the construction.

Reading. Two opposite lessons. The savings *shape* matters and survives the annealing: the best cell after SA is $\lambda = 1.2$, $\mu = 0.2$, worth -0.042% against $\lambda = 1$, $\mu = 0$. The savings *list*, by contrast, does not survive at all: truncating it to the 5 nearest neighbours costs $+7.65\%$ on the raw construction and -0.019% once the annealing has run. That is a useful lever for large n , where Section 7 showed the $O(n^2)$ list is the dominant cost — the annealing repairs what the truncation breaks. `--2opt` is likewise absorbed: it improves the construction and changes nothing afterwards.

15 Do the knobs combine?

What is tested. Tuning one knob at a time says nothing about tuning several at once. These runs take three plausible changes, apply them individually and together at equal SA budget, and compare against the stock defaults. `all_r8` additionally splits the same budget over 8 restarts.

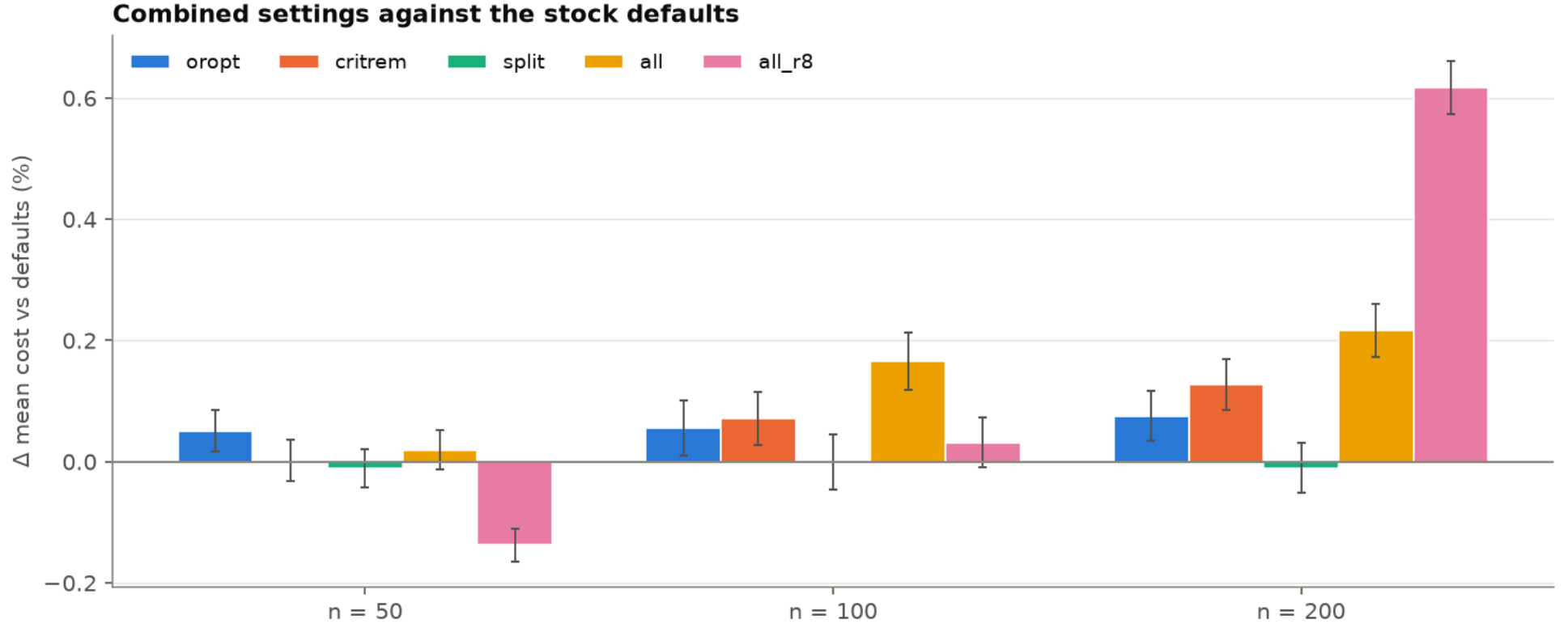


Figure 16: Candidate combinations against the stock defaults at equal SA budget, with 95 % CIs.

Table 32: Combinations against the stock defaults.

<i>n</i>	variant	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
50	or-opt enabled	10.27561	+0.051	[+0.017, +0.085]	286/359	5.04
100	or-opt enabled	15.51793	+0.055	[+0.010, +0.101]	471/511	5.17
200	or-opt enabled	21.98240	+0.076	[+0.034, +0.117]	458/542	5.31
50	pick-crit rem	10.27062	+0.002	[-0.033, +0.037]	330/307	4.87
100	pick-crit rem	15.52041	+0.071	[+0.027, +0.116]	457/528	4.98
200	pick-crit rem	21.99383	+0.128	[+0.086, +0.170]	418/582	5.07
50	Split both + every 100,000	10.26933	-0.010	[-0.042, +0.021]	344/293	5.04
100	Split both + every 100,000	15.50921	-0.001	[-0.046, +0.044]	509/469	5.15

n	variant	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
200	Split both + every 100,000	21.96348	-0.011	[-0.052, +0.031]	512/487	5.24
50	the three combined	10.27238	+0.019	[-0.013, +0.052]	327/325	5.04
100	the three combined	15.53502	+0.166	[+0.118, +0.213]	423/559	5.14
200	the three combined	22.01323	+0.216	[+0.172, +0.260]	387/613	5.33
50	the three, over 8 restarts	10.25629	-0.137	[-0.165, -0.110]	450/185	5.11
100	the three, over 8 restarts	15.51423	+0.032	[-0.010, +0.073]	458/528	5.18
200	the three, over 8 restarts	22.10146	+0.618	[+0.573, +0.662]	184/816	5.33

n	default	newall	Δ (%)	95 % CI	win/loss	Δ at $1\times$ (%)
100	15.46965	15.43432	-0.228	[-0.258, -0.198]	684/258	-0.329
200	21.88892	21.78901	-0.456	[-0.492, -0.421]	808/192	-0.556

Table 33: The $3\times$ rung (30,000,000 steps): the same comparison at three times the tier’s budget, with the $1\times$ delta beside it. If the two columns agree, the ordering has settled; if the $3\times$ delta is still shrinking, the budget has not yet stopped mattering.

Reading. The combination is worse than its parts: `all` loses to the defaults at every n tested ($n=50$, $n=100$, $n=200$), even though one of its three ingredients helps on its own. Gains found one knob at a time do not add up, which is exactly why the recommendation in Section 16 is confirmed by re-running it rather than assembled on paper.

16 The configuration to use

A knob enters the command below only if its best setting beat *its own* default significantly — both the 95 % CI excluding zero and the sign test rejecting at 5 %. Everything else stays at the default, deliberately: a knob whose interval straddles zero has not earned a change.

knob	setting	Δ (%)	95 % CI	win/loss	flags
new operators	<code>mix_1-0-1-0-1-0.05</code>	-0.335	[-0.376, -0.294]	692/278	<code>--ops 1,0,1,0,1,0.05</code>
schedule	<code>a0.001_d1</code>	-0.199	[-0.240, -0.158]	607/356	<code>--t-decades 1</code>
restarts (equal budget)	<code>iso_R16</code>	-0.158	[-0.188, -0.128]	562/389	<code>--restarts 16</code>
partner tournament	<code>pick2_2</code>	-0.067	[-0.110, -0.024]	536/437	<code>--pick2 2</code>
racing	<code>margin_0.0005</code>	-0.066	[-0.082, -0.049]	494/293	<code>--race 0.0005</code>

Table 34: Knobs that earned their place, with the improvement each showed on its own.

knob	best setting tried	Δ (%)	95 % CI	win/loss	why not
operators	sub_1110	+0.000	[+0.000, +0.000]	0/0	no significant improvement
or-opt length	ormax_8	-0.011	[-0.057, +0.035]	513/466	no significant improvement
neighbourhood	n100_K20	+0.000	[+0.000, +0.000]	0/0	no significant improvement
vertex choice	T2_raw	-0.019	[-0.062, +0.024]	496/473	no significant improvement
Split	mend_e100000	-0.030	[-0.071, +0.012]	492/467	no significant improvement
savings shape	sa_l1.2_m0.2	-0.042	[-0.085, +0.000]	504/440	no significant improvement
initialisation	n100_cw_s10000000	+0.000	[+0.000, +0.000]	0/0	no significant improvement
partner bias	vrank1_K20	+0.000	[+0.000, +0.000]	0/0	no significant improvement
relocate side	side_long	-0.020	[-0.063, +0.023]	504/467	no significant improvement

Table 35: Knobs left at their default, and why. Note the distinction: most of these simply showed no significant improvement, but a knob can also be dropped while being significant on its own, if it is inert in the company it would keep.

One caveat the table above cannot express. Racing was worth -0.066% on its own and is kept — and so is `--restarts`, which is what makes it useful: the two only pay together. At $n = 100$ and equal total budget, the recommended options with a single start reach 15.45834 (-0.329% vs the default), and the same options with 8 starts and racing reach 15.42787 (-0.525%). If you are going to spend the budget on restarts at all, turn racing on; the sweep’s one-knob-at-a-time frame simply has no way to say so.

16.1 The command

Written out in full, defaults included, so the run does not depend on what `cw`’s built-in defaults happen to be:

```
./cw --bundle data/cvrp_100.cvrpb \
  --init cw \
  --knn 0 --lambda 1 --mu 0 \
  --sa-steps 625000 --ops 1,0,1,0,1,0.05 --or-max 3 \
  --t-accept 0.001 --t-decades 1 \
  --sa-knn 20 --pick 2 --pick-crit 1b --pick-eps 0.3 \
  --vrank 1 --pick2 2 --reloc-side coin \
  --restarts 16 --cw-rand perturb --cw-alpha 0.03 \
  --race 0.0005 --race-at 0.25 --pair 0 \
  --split off --split-every 0 --split-tour both
```

On generated instances instead of a bundle, swap the first line for `--random -n 100 -m 1000 --seed 42`:

```
./cw --random -n 100 -m 1000 --seed 42 \
  --init cw \
  --knn 0 --lambda 1 --mu 0 \
```

```

--sa-steps 625000 --ops 1,0,1,0,1,0.05 --or-max 3 \
--t-accept 0.001 --t-decades 1 \
--sa-knn 20 --pick 2 --pick-crit lb --pick-eps 0.3 \
--vrank 1 --pick2 2 --reloc-side coin \
--restarts 16 --cw-rand perturb --cw-alpha 0.03 \
--race 0.0005 --race-at 0.25 --pair 0 \
--split off --split-every 0 --split-tour both

```

Only `--ops`, `--t-decades`, `--pick2`, `--restarts` and `--race` differ from the stock defaults (`--ops` was 1,1,1,0,0,0, `--t-decades` was 2, `--pick2` was 1, `--restarts` was 1 and `--race` was 0). Every other value above is what `cw` would have used anyway, written out so the run is reproducible against a future version whose defaults have moved.

16.2 What the changed options do

`--ops` — the operator mixture (Sections 4 and 5). Positions 5 and 6 are `swap*` and route opening, both zero by default. `swap*` is the only non-elementary move in the solver, $O(L_1 + L_2)$ per draw against $O(1)$ for everything else, so it is the one option here whose equal-step gain and equal-time gain genuinely differ; see the wall-time panel of Section 5 for the comparison that counts.

`--t-decades` (default 2) — the width of the temperature schedule. The annealing does not take T_0 as an argument: `calibrate_T0` samples worsening moves on the instance itself and solves for the T_0 at which a fraction `--t-accept` of them would be accepted (0.1% by default). From there the temperature decays geometrically to $T_{\text{end}} = T_0 \cdot 10^{-D}$, one factor $\alpha = 10^{-D/(\text{steps}-1)}$ per step, with $D = \text{--t-decades}$. So D is *how many orders of magnitude the temperature falls over the run*: $D = 2$ ends a hundred times colder than it started, $D = 1$ only ten times colder. Lowering D keeps the search warm for longer; the reading is that the default schedule freezes early and spends its last steps at temperatures where essentially nothing is accepted. It is budget-dependent by construction.

`--pick2` (default 1) — a tournament of size T among candidate partners, keeping the one of largest regret. Unlike `--vrank` this costs a memory read per candidate.

`--restarts` — how many independent Clarke & Wright constructions are annealed, the best kept (Section 8). Here the comparison is always at equal *total* budget, so more restarts means proportionally fewer steps each.

`--race` (default off) — budget redistribution between restarts (Section 12). At `--race-at` of its budget a start is compared with the best finished start *at the same point of its trajectory*, and abandoned if it is behind by more than the margin. The unspent steps go to the starts that follow, and the remainder polishes the best solution found. The total budget is unchanged; only its allocation is. It does nothing with a single start.

These are conditional on the budget and on the instance family. Everything above was tuned at `--sa-steps` = 10,000,000 on uniform points in a square. The temperature schedule in particular trades off against the budget by construction — re-run the `temp` study if you change it substantially — and the savings parameters were fitted to one geometry.

16.3 Confirmation

The combination is re-run from scratch, because Section 15 showed that knobs tuned separately do not add up. It is checked on the sweep’s own instances and on 5 seeds the sweep never saw — the difference between the two blocks is the selection bias made visible.

instances	n	default	recommended	Δ (%)	95 % CI	win/loss
sweep seeds (in-sample)	50	10.27040	10.24224	-0.274	[-0.303, -0.246]	586/38
sweep seeds (in-sample)	100	15.50934	15.41868	-0.585	[-0.622, -0.547]	864/106
sweep seeds (in-sample)	200	21.96580	21.79890	-0.760	[-0.799, -0.720]	926/74
fresh seeds	50	10.34391	10.31249	-0.304	[-0.335, -0.272]	561/46
fresh seeds	100	15.51274	15.42276	-0.580	[-0.617, -0.543]	868/105
fresh seeds	200	21.97188	21.80800	-0.746	[-0.783, -0.709]	919/81

Table 36: The recommended configuration against the stock defaults, at equal SA budget. The fresh-seed rows are the honest estimate. The recommendation uses 16 restarts of 625,000 steps against the default’s single run of 10,000,000, so both sides spend the same total.

17 Caveats

- **One instance family.** Uniform coordinates and uniform demands. Nothing here is evidence about clustered or real-world instances; run `tools/fetch_neuopt.py` and point the sweep at `--bundle` for that.
- **Two budgets, not a curve.** Every grid is run at 10^5 and at 10^7 annealing steps (Section 2), which is enough to say whether a recommendation transfers but not to fit how it varies. The schedule knob in particular is budget-dependent by construction — the geometric ratio is $(T_{\text{end}}/T_0)^{1/(S-1)}$ — so at a third budget it would have to be re-measured again.
- **The two tiers use different instance counts.** $m = 1000$ per seed at 10^5 steps and $m = 200$ at 10^7 , to keep the total compute bounded. Pooled over the seeds both tiers carry ≥ 1000 paired instances, so their CIs are comparable, but the 10^7 tier is the looser of the two.
- **Selection bias.** The overview picks each knob’s best setting *because* it scored lowest. The fresh-seed rows in Section 16 are the only bias-free numbers in this document.
- **--sa-knn is clamped** to $n - 1$ (`cw.c:1675`), so at $n = 20$ the settings $K \geq 20$ are the same run; and at $K = 0$ the vertex-selection rule is silently forced to uniform (`cw.c:1676`) while the header still prints the requested `--pick`. That is a reporting bug worth fixing in `cw`.
- **One machine, for the wall-time results.** The iso-time reading in Section 5 and the interleaving speed-up in Section 12 are the only conclusions here that depend on the hardware. Both were measured on one laptop with all threads busy; the cost ratios will move on a machine with a different cache hierarchy, and the interleaving result in particular is a cache effect by construction.
- **Knobs that need each other are invisible to the frame.** Every knob is swept with everything else at its default, so a knob that only pays in the presence of another cannot show up. Racing is the clear case — significant on its own measurement, dropped from the recommendation, and worth having as soon as `--restarts` is raised. Section 15 is the only part of this document that looks at combinations at all.
- **Where this disagrees with its source.** `--vrank` and `--sa-knn 30` were ported from an implementation that reports them as a win, and here they lose consistently across the whole grid. The likely cause is that the other implementation shares one $K = 30$ neighbour list between the savings construction and the annealing while this one builds exact savings up to $n = 1500$, so the two are not the same baseline. This sweep can say the setting does not pay *here*; it cannot say the original measurement was wrong.